

Personalización de ranking para e-commerce reseller: un estudio empírico de embeddings comerciales, representación multi-vector del usuario, reranking contextual y ranking multi-objetivo

Yosvany Castro

2026

Resumen

Este trabajo eleva el pipeline de personalización de un e-commerce reseller (reventa de catálogo Amazon/AliExpress sin stock físico, donde cada llamada al agregador tiene costo real) desde una heurística de relevancia única a un sistema de ranking de dos etapas evaluado con rigor. Sobre un simulador de marketplace con verdad de fondo conocida y un arnés de evaluación con holdout temporal, se estudian empíricamente cuatro contribuciones: (1) embeddings comerciales (texto, Prod2Vec, híbrido, two-tower, late-interaction, contextualizado), hallando que capturan relevancia pero no complementariedad; (2) representación multi-vector del usuario con un modelo explícito de regalo, que supera al vector único sobre todo en sesiones de regalo; (3) un pool de candidatos multi-fuente con cuatro familias de reranker, que cambian el conjunto recuperado pero no superan a la fusión RRF en relevancia pura; y (4) ranking multi-objetivo, cuya frontera de Pareto permite negociar explícitamente relevancia y revenue. Se reportan los hallazgos negativos con la misma honestidad que los positivos, y se diseña —sin ejecutarlo— un piloto A/B para producción.

Índice

1	Introducción	3
1.1	Contexto y problema	3
1.2	Crítica de partida	3
1.3	Contribuciones	4
1.4	Enfoque metodológico	5
1.5	Estructura del documento	5
2	Estado del arte	5
2.1	Representación del usuario	6
2.2	Embeddings de recuperación	6
2.3	Fusión y diversidad	6
2.4	Cross-sell y complementariedad	7
2.5	Reranking con modelos de lenguaje	7
2.6	Ranking multi-objetivo	7
2.7	Evaluación y estimación off-policy	8
2.8	Síntesis	8
2.9	Referencias	8

3	Metodología	9
3.1	El simulador de marketplace con verdad de fondo	10
3.2	Por qué el ground-truth es imprescindible	12
3.3	El arnés de evaluación	12
3.4	Disciplina de datos reales: sin mocks	13
3.5	Diseño de las fases experimentales	14
3.6	Advertencia de escala y validez externa	16
3.7	Reproducibilidad y separación de código	16
4	Embeddings comerciales (F1)	17
4.1	Motivación y pregunta de investigación	17
4.2	Diseño experimental: comparación justa entre espacios vectoriales	17
4.3	Resultados	18
4.4	Hallazgos	18
4.5	Síntesis del capítulo	19
5	Usuario multi-vector y modelo de regalo (F2)	20
5.1	Motivación y pregunta de investigación	20
5.2	Diseño: un usuario como distribución de intereses	20
5.3	Resultados	21
5.4	Discusión y hallazgos	22
6	Generación de candidatos y reranking (F3)	23
6.1	Motivación y diagnóstico previo	23
6.2	Diseño del sistema F3	23
6.3	Resultados	24
6.4	Discusión y hallazgos	27
7	Ranking multi-objetivo (F4)	27
7.1	Motivación: del ranking de relevancia a la negociación de objetivos	27
7.2	Diseño experimental	28
7.3	Frontera de Pareto y barrido de λ	29
7.4	Hallazgo central: el trade-off es real y ajustable	29
7.5	Honestidad metodológica: límites del punto de operación	30
7.6	Síntesis	31
8	Discusión	31
8.1	Qué funcionó: convergencia entre fases	31
8.2	Los hallazgos negativos como contribución científica	33
8.3	Limitaciones del enfoque	35
8.4	Amenazas a la validez y mitigaciones	36
9	Conclusión y trabajo futuro	37
9.1	Conclusión	37
9.2	Trabajo futuro	38
10	Plan de piloto A/B (diseño, no ejecutado)	39
10.1	Justificación: por qué el piloto es la validación que falta	39
10.2	Qué se promueve a producción bajo feature flags	40

10.3	Diseño experimental	41
10.4	Guardrails y rollback automático	43
10.5	Riesgos y mitigaciones	44
10.6	Rollout por fases	45
10.7	Resumen del protocolo	46

1 Introducción

1.1 Contexto y problema

El e-commerce en Cuba presenta restricciones estructurales que condicionan toda decisión de ingeniería: acceso limitado a logística propia, infraestructura de pago fragmentada y un catálogo que debe servirse desde intermediarios externos. El sistema objeto de este trabajo opera como reseller puro de catálogos de Amazon y AliExpress: no mantiene stock físico y cada consulta a los agregadores de producto tiene un costo directo en producción. Esta realidad económica eleva la minimización de llamadas fallidas al agregador a prioridad arquitectónica de primer orden, al mismo nivel que la relevancia de los resultados.

Sobre esta base, el pipeline de personalización de partida asignaba a cada ítem del catálogo una puntuación escalar derivada de un único criterio de relevancia. Formalmente, el peso del único objetivo era $\lambda_{\text{relevancia}} = 1$, lo que equivale a asumir que la relevancia percibida por el usuario agota por completo los intereses del negocio y que el perfil de intereses del usuario puede representarse con un solo vector. Ambas asunciones son convenientes pero cuestionables, y su cuestionamiento sistemático constituye el punto de partida de este trabajo.

1.2 Crítica de partida

Una auditoría adversarial del pipeline, documentada en `docs/handoff-audit-reranker-pipeline.md` y en `docs/superpowers/reports/2026-05-15-fase-3c-audit-razones.md`, identificó tres debilidades estructurales en el diseño de partida.

La primera era que el reranker LLM no cambiaba el conjunto de ítems recuperados. Operaba sobre un pool de treinta candidatos generados por MMR y devolvía prácticamente los mismos diez ítems que el paso previo: reordenaba sin renovar. La causa era estructural, no de calidad del modelo: un pool tan reducido deja al reranker sin material diferencial respecto al retriever, de modo que no puede elevar un ítem correcto que el retriever nunca incluyó.

La segunda debilidad era que el perfil de usuario basado en un único vector colapsaba ante las sesiones de regalo. Cuando un comprador adquiere artículos para una persona con gustos radicalmente distintos a los suyos —un hombre adulto comprando artículos de bebé—, esos eventos se incorporan al perfil permanente y contaminan las recomendaciones futuras de autoconsumo. Un vector único no tiene mecanismo para separar el interés efímero del interés permanente.

La tercera debilidad era conceptual: la similitud coseno de embeddings de texto no captura la complementariedad comercial. Un ratón inalámbrico y su alfombrilla, una funda y el teléfono que protege, pueden estar en categorías léxicamente distantes y, sin embargo, comprarse juntos de forma consistente. La distancia coseno en el espacio de embeddings de texto no puede recuperar esa relación funcional porque no fue entrenada sobre señal de compra conjunta.

El veredicto cualitativo de la auditoría fue más profundo que la lista de fallos técnicos: planteó la hipótesis de que un “e-commerce normal bien hecho” —catálogo ordenado por popularidad,

búsqueda léxica y filtros de categoría— ya capturaba la mayor parte del valor para el usuario final, de modo que cualquier capa de personalización profunda debía justificarse con evidencia empírica rigurosa, y no asumirse como mejora por su mera existencia. Esta es la pregunta motivacional que el programa de tesis se propuso responder: ¿cuándo y cuánto ayuda la personalización, y dónde no ayuda?

1.3 Contribuciones

El programa de tesis se articula en cuatro fases de experimentación (F1–F4), precedidas por la construcción de la infraestructura de evaluación (F0). Cada fase prueba una idea empíricamente, mide su lift y reporta sus límites con la misma honestidad que sus victorias. Las tres ideas que guiaron el programa son las siguientes.

Primera contribución: el usuario como distribución multi-modo con modelo explícito de regalo. Se abandona la representación de usuario como punto único en el espacio de embeddings y se adopta un modelo de distribución de intereses inspirado en PinnerSage: los intereses del usuario forman clústeres, cada uno representado por su medoide, y durante la recuperación cada modo emite candidatos de forma proporcional a su peso relativo. Sobre esta base se añade un eje ortogonal al modelado de intereses: la detección de intención de regalo y la construcción de un vector de destinatario efímero, que sustituye temporalmente al perfil del comprador durante la sesión sin alterar su historial permanente. Esta factorización comprador $\times$ destinatario no está modelada explícitamente en la literatura de multi-interés consultada y constituye una contribución original.

Segunda contribución: la relación comercial no es proximidad lingüística. La complementariedad entre productos —la relación que hace que una funda sea el complemento natural de un teléfono— no es recuperable por similitud coseno en el espacio de texto. Este trabajo modela la complementariedad con co-ocurrencia de compra ponderada por *Normalized Pointwise Mutual Information* (NPMI), y la integra como fuente independiente en un pool multi-fuente de candidatos que se fusiona mediante *Reciprocal Rank Fusion* (RRF). El resultado es que ítems complementarios que ningún retriever semántico hubiera incluido entran al pool y son accesibles al reranker.

Tercera contribución: el ranking como negociación multi-objetivo. El score de relevancia pura es un caso particular de una familia más general de funciones de ranking donde distintos objetivos del negocio coexisten bajo pesos ajustables. El trabajo implementa una combinación lineal convexa con cuatro objetivos —relevancia, revenue esperado, margen bruto y diversidad de vendedor— y estudia la frontera de Pareto entre relevancia y revenue. La principal lección es que esta frontera existe y no es trivial: el operador puede mover explícitamente el punto de operación a lo largo de ella sin necesidad de reentrenamiento.

Como contribución habilitadora se estudia también empíricamente la elección de embedding comercial (F1): seis familias de representación de producto —texto puro, Prod2Vec comportamental, modelo híbrido, two-tower con corrección logQ, late-interaction y modelo contextualizado— se evalúan sobre los mismos 1 999 ítems y bajo el mismo arnés de evaluación, revelando que ningún embedding recupera complementos por similitud coseno, lo que motiva directamente la contribución de co-ocurrencia.

1.4 Enfoque metodológico

La atribución rigurosa del lift a cada componente requiere conocer la verdad. Con datos observacionales de producción, la preferencia real del usuario se confunde con la exposición y la popularidad. Este trabajo construye un simulador de marketplace con verdad de fondo conocida: los gustos latentes del usuario, las relaciones de complementariedad y la siguiente compra que realizaría son generados por el sistema y conocidos de antemano. Sobre ese suelo firme se construye un arnés de evaluación con holdout temporal que permite atribuir el lift de cada componente de forma inequívoca y reportar honestamente dónde un método no ayuda. En una frase: el simulador con ground-truth más el arnés de evaluación riguroso permiten separar lo que funciona de lo que meramente parece funcionar.

1.5 Estructura del documento

El documento sigue la secuencia lógica del programa de tesis:

- **Estado del arte** (02-related-work.md): revisión de la literatura sobre representación multi-vector del usuario, embeddings de recuperación, complementariedad cross-sell, reranking con modelos de lenguaje, ranking multi-objetivo y evaluación off-policy. Se posiciona cada contribución respecto a la literatura.
- **Metodología** (03-metodologia.md): descripción del simulador de marketplace, el arnés de evaluación, la disciplina de no-mocks y el diseño de fases como experimentos acumulativos.
- **F1 — Embeddings comerciales** (04-f1-embeddings.md): comparación de seis familias de representación de producto; hallazgo de que los embeddings no recuperan complementos por similitud coseno.
- **F2 — Usuario multi-vector y modelo de regalo** (05-f2-multivector.md): modelo de distribución de intereses con medoides adaptativos y vector efímero de destinatario para sesiones de regalo.
- **F3 — Pool multi-fuente y reranking** (06-f3-rerank.md): diseño del pool de candidatos de cuatro fuentes, evaluación de cuatro familias de reranker y hallazgo de que ningún reranker supera a la fusión RRF en relevancia pura.
- **F4 — Ranking multi-objetivo** (07-f4-multiobjetivo.md): implementación de la combinación lineal convexa de objetivos, estudio de la frontera de Pareto entre relevancia y revenue, y limitaciones de la aproximación a la probabilidad de compra.
- **Discusión** (08-discusion.md): integración de los hallazgos de F1–F4, limitaciones del enfoque sintético y consideraciones de transferibilidad a datos reales de producción.
- **Conclusión y trabajo futuro** (09-conclusion-trabajo-futuro.md): síntesis de contribuciones, hallazgos negativos que merecen atención y líneas abiertas.
- **Plan de piloto A/B** (10-plan-piloto.md): diseño sin ejecutar de un experimento controlado en producción para validar el lift observado offline con tráfico real.

2 Estado del arte

Este capítulo revisa el estado del arte en las áreas que fundamentan el programa de tesis: representación del usuario, embeddings de recuperación, fusión y diversidad, complementariedad

cross-sell, reranking con modelos de lenguaje, ranking multi-objetivo y evaluación off-policy. Para cada área se posiciona la contribución de este trabajo respecto a la literatura existente.

El aporte central de esta tesis es la integración de representación multi-vector del usuario, co-ocurrencia para cross-sell, reranking sobre un pool amplio de candidatos y ranking multi-objetivo en un **pipeline único**, evaluado con ground-truth sintético de verdad conocida. A diferencia de trabajos que optimizan cada componente de forma aislada, aquí se miden las interacciones entre componentes —incluyendo los hallazgos negativos— como parte explícita de la contribución.

2.1 Representación del usuario

La representación del usuario es la base de cualquier sistema de recomendación personalizado. PinnerSage [1] propone representar a cada usuario mediante varios vectores (modos de interés) obtenidos por clustering Ward de su historial, con cada modo caracterizado por su medoide y su peso relativo; esta arquitectura multi-vector supera a los perfiles de vector único cuando el usuario tiene intereses genuinamente ortogonales. MIND [2] adopta un enfoque similar basado en cápsulas de interés dinámicas, demostrando que la diversidad interna del perfil de usuario mejora la cobertura del recall. Trabajos más recientes de Pinterest [3] extienden esta línea modelando intereses implícitos y explícitos de forma conjunta, con señales comportamentales de alta frecuencia. Este trabajo aplica el núcleo de PinnerSage (representación multi-vector con medoides adaptativos, vía clustering aglomerativo de enlace promedio con distancia coseno) y lo extiende con un **eje destinatario/regalo**: cuando la sesión revela intención de regalo, el perfil del destinatario reemplaza temporalmente el perfil propio del comprador, evitando que el historial de regalos contamine los modos permanentes del usuario. Esta factorización usuario $\times$ destinatario no aparece modelada explícitamente en la literatura de multi-interés consultada, y constituye una contribución original defendible.

2.2 Embeddings de recuperación

Los modelos two-tower con corrección logQ [4] permiten entrenar embeddings de ítems y usuarios a partir de feedback implícito con exposición sesgada, corrigiendo la distorsión por popularidad que introduce el sistema de producción. Item2Vec y Prod2Vec [5] extienden la idea de Word2Vec al catálogo de productos, aprendiendo representaciones comportamentales desde co-clics o co-compras sin necesidad de atributos textuales. La familia ColBERT/ColBERTv2 y su extensión multilingüe Jina-ColBERT-v2 [6] adopta la interacción tardía (late interaction) con puntuación MaxSim por fragmento, logrando la precisión de los cross-encoders con un coste de inferencia más próximo a los bi-encoders. El modelo **voyage-context-3** [7] produce embeddings contextualizados por fragmento, útiles cuando el texto del ítem contiene información en múltiples bloques semánticamente heterogéneos. Este trabajo implementa las cinco estrategias (texto plano, Prod2Vec, híbrido, two-tower y late-interaction ColBERT) bajo una interfaz común **Embedder** y las compara de forma controlada sobre el mismo arnés de evaluación, encontrando que todas capturan relevancia pero ninguna captura complementariedad, lo que motiva el uso de co-ocurrencia NPMI como fuente separada.

2.3 Fusión y diversidad

Reciprocal Rank Fusion (RRF) [8] propone combinar múltiples listas de resultados ponderando el rango recíproco de cada ítem en cada lista, sin necesidad de normalizar los scores de diferentes

retrievers. Su sencillez y robustez lo han convertido en el estándar de facto para la fusión de candidatos multi-fuente. Maximal Marginal Relevance (MMR) [9] introduce un criterio de selección greedy que balancea relevancia e información marginal, reduciendo la redundancia en el conjunto final de resultados. Este trabajo utiliza RRF como mecanismo de fusión del pool multi-fuente en F2 y F3, y adopta una selección estilo MMR para el término de diversidad marginal en el scorer multi-objetivo de F4. Un hallazgo importante es que RRF sobre el pool multi-fuente resulta ser un baseline muy competitivo: los rerankers LLM y LTR consiguen cambiar el conjunto recuperado (set-change@10 medible) pero **no superan a RRF en nDCG**, lo que constituye un hallazgo negativo honesto de este trabajo.

2.4 Cross-sell y complementariedad

La asociación de productos complementarios mediante co-ocurrencia ponderada, medida con NPMI (Normalized Pointwise Mutual Information) [10], captura relaciones de compra conjunta que **no son de proximidad lingüística** sino de comportamiento comercial: un cable y un cargador pueden ser complementarios aunque sus textos no sean similares. El NPMI permite distinguir complementos verdaderos de pares que simplemente co-ocurren por popularidad, dado que normaliza por las frecuencias marginales. Este trabajo confirma empíricamente, a partir de seis familias de embeddings, que ningún espacio vectorial basado en texto o comportamiento individual recupera complementos con complement-recall@10 superior a 0,001. En consecuencia, la co-ocurrencia NPMI se incorpora como **fuentes explícitas del pool de candidatos y como feature del reranker**, no como señal de proximidad vectorial. Este diseño es coherente con la distinción conceptual entre similitud semántica y asociación estadística de compra.

2.5 Reranking con modelos de lenguaje

RankGPT [11] y su sucesor RankZephyr demuestran que los modelos de lenguaje instrucciones pueden puntuar o reordenar documentos de forma listwise sin entrenamiento adicional de reranking, obteniendo ganancias en benchmarks de recuperación de información. LLM4Rerank [12] sistematiza el paradigma, explorando estrategias de prompting y la interacción entre el tamaño del pool y la calidad del reranking. REARANK [13] avanza hacia reranking razonado: el modelo genera una cadena de justificación antes de producir el ranking final. Este trabajo sitúa el reranking LLM listwise (DeepSeek, sin GPU) como una de las cuatro familias de reranker comparadas en F3, junto con LTR aprendido, cross-encoder MaxSim (aproximación de E4 sin GPU) y baselines MMR/RRF. El hallazgo principal es que **el reranker LLM no estaba roto en el sistema original**, sino hambriento: con un pool de 10 candidatos no había de dónde elegir; con 100–300 candidatos sí cambia el conjunto. Sin embargo, sigue sin superar a RRF en nDCG, sugiriendo que los gains del reranking LLM en benchmarks de IR pueden no transferirse directamente a catálogos de e-commerce con distribuciones de popularidad sesgadas.

2.6 Ranking multi-objetivo

El ranking multi-objetivo para sistemas de recomendación ha sido formulado desde la perspectiva de la optimización de Pareto [14], donde ninguna configuración de pesos domina a todas las demás en todos los objetivos simultáneamente. Airbnb [15] documenta un caso industrial de MOO constreñido para balancear satisfacción de huéspedes y salud del inventario, demostrando que los trade-offs son cuantificables y operacionalizables. MOO-by-distillation [16] propone aprender directamente un ranker multi-objetivo mediante destilación desde múltiples rankers especializados. Los bandits de diversidad [17] abordan el problema de forma online, aprendiendo políticas que

intercalan objetivos de relevancia y diversidad a lo largo del tiempo. Este trabajo adopta el enfoque offline: un scorer lineal $s(p|u) = \sum_k \lambda_k \cdot f_k(p, u)$ con un barrido determinista de λ sobre seis features normalizadas (relevancia, margen, probabilidad de conversión, diversidad marginal, novedad, fairness de vendedores). El resultado es la **frontera de Pareto** entre relevancia y revenue, con la prueba cuantitativa de que “subir revenue cuesta X de relevancia”. Esto convierte el ranking de un score implícito ($\lambda_{\text{relevancia}} = 1$, todo lo demás 0, que es lo que hace el sistema hasta F3) en una **decisión de negocio explícita y medible**.

2.7 Evaluación y estimación off-policy

nDCG (Normalized Discounted Cumulative Gain) es la métrica estándar para evaluar calidad de ranking; trabajos recientes [18] discuten su uso como métrica off-policy cuando los logs del sistema de producción no son exploratorios, destacando el sesgo de exposición que introduce el sistema de producción en las estimaciones. La estimación off-policy de generadores de candidatos [19] proporciona marcos formales para estimar el valor de una política nueva a partir de logs históricos sin desplegar el sistema. AlignUSER [20] propone world models impulsados por LLM para simular el comportamiento del usuario y evaluar sistemas de recomendación antes del despliegue en producción. Este trabajo aborda las limitaciones de la evaluación off-policy con datos de producción mediante el uso de un **simulador con ground-truth conocido**: al generar usuarios sintéticos con estado latente conocido (gustos multimodales, destinatarios, sensibilidad al precio, grafo de complementariedad) y comportamiento generativo calibrado, las métricas de evaluación offline son comparables entre familias de métodos sin el sesgo de exposición del sistema real. El simulador no reemplaza la evaluación online, pero permite falsificar hipótesis de forma controlada y es coherente con las tendencias de simulación descritas en la literatura de world models para recomendación.

2.8 Síntesis

Ninguno de los trabajos revisados combina simultáneamente los cuatro componentes del pipeline de esta tesis: representación multi-vector con eje destinatario/regalo, co-ocurrencia NPMI como fuente de candidatos cross-sell, reranking sobre un pool amplio comparando cuatro familias, y ranking multi-objetivo con frontera de Pareto explícita entre relevancia y revenue. Más importante: ninguno los evalúa en un sistema real con restricciones de coste (sin GPU, sin créditos ilimitados de LLM) y reporta los hallazgos negativos —que los embeddings no capturan complementariedad, que el reranker LLM no supera al RRF— con la misma honestidad que los positivos. Esos hallazgos negativos son parte de la contribución.

2.9 Referencias

- [1] Pal, A. et al. “PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest.” *KDD*, 2020.
- [2] Li, C. et al. “Multi-Interest Network with Dynamic Routing for Recommendation at Tmall (MIND).” *CIKM*, 2019.
- [3] Pinterest Engineering. “Modeling Implicit and Explicit User Interests at Pinterest.” *KDD*, 2025.

- [4] Yi, X. et al. “Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations (two-tower con corrección logQ).” *RecSys*, 2019.
- [5] Barkan, O. and Koenigstein, N. “Item2Vec: Neural Item Embedding for Collaborative Filtering (Prod2Vec).” *MLSP*, 2016.
- [6] Santhanam, K. et al. “ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction.” *NAACL*, 2022. Incluye extensión multilingüe Jina-ColBERT-v2.
- [7] Voyage AI. “**voyage-context-3**: Contextualized chunk embeddings.” Documentación técnica, 2024.
- [8] Cormack, G. V., Clarke, C. L. A. and Buettcher, S. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” *SIGIR*, 2009.
- [9] Carbonell, J. and Goldstein, J. “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries.” *SIGIR*, 1998.
- [10] Church, K. W. and Hanks, P. “Word Association Norms, Mutual Information, and Lexicography.” *Computational Linguistics*, 1990. Fundamento estadístico del NPMI aplicado a co-ocurrencia de productos.
- [11] Sun, W. et al. “Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents (RankGPT).” *EMNLP*, 2023.
- [12] “LLM4Rerank: Large Language Models as Rerankers.” *WWW*, 2025.
- [13] “REARANK: Reasoning-Enhanced Listwise Reranking.” Preprint, 2025.
- [14] Ehrgott, M. *Multicriteria Optimization*. Springer, 2005. Marco teórico de la frontera de Pareto aplicado a ranking multi-objetivo.
- [15] Airbnb Engineering. “Constrained Multi-Objective Optimization for Ranking.” Blog técnico / conferencia, 2024.
- [16] “Multi-Objective Ranking via Distillation from Specialized Rankers (MOO-by-distillation).” Preprint, 2024.
- [17] Yue, Y. and Joachims, T. “Interactively Optimizing Information Retrieval Systems as a Dueling Bandits Problem.” *ICML*, 2009. Base teórica para bandits de diversidad en recomendación.
- [18] Saito, Y. “nDCG as an Off-Policy Evaluation Metric.” *RecSys*, 2023.
- [19] “Off-Policy Evaluation of Candidate Generators in Recommender Systems.” *RecSys*, 2025.
- [20] “AlignUSER: World Models with LLM for User Simulation in Recommendation.” Preprint, 2026.

3 Metodología

Este capítulo describe el aparato experimental que sustenta el programa empírico. La metodología se articula en cuatro pilares: el simulador de marketplace con verdad de fondo conocida (ground-truth), el arnés de evaluación que explota esa verdad para medir con rigor, la disciplina de no-mocks que garantiza la fidelidad al sistema real, y el diseño de fases F0–F4 como experimentos sucesivos

que se construyen uno sobre el anterior. Todos los detalles de ingeniería que se mencionan aquí están especificados en los documentos de diseño citados al pie de cada sección.

3.1 El simulador de marketplace con verdad de fondo

3.1.1 Motivación del enfoque sintético

Evaluar un sistema de personalización con datos puramente observacionales plantea un problema estructural: no se puede saber si el sistema recomienda lo correcto porque el usuario realmente lo prefería, o porque el historial de comportamiento ya estaba sesgado hacia los ítems más visibles. La confusión entre popularidad, exposición y preferencia real es endémica en los logs de producción. Adicionalmente, conceptos como la complementariedad comercial (que una funda y un teléfono son complementos, no porque suenen parecido sino porque se usan juntos) o la intención de regalo (que el usuario compra para alguien con gustos diferentes a los suyos) son imposibles de aislar sin etiquetar manualmente a gran escala.

Para evitar estas limitaciones, el programa construye un **simulador de marketplace** en el que la “verdad” es generada por el propio sistema: se conocen el gusto real de cada usuario, la intención de cada sesión, las relaciones de complementariedad entre productos y la siguiente compra que cada usuario realizaría. Esto hace posible (a) un holdout temporal sin fuga de información y (b) ablations que atribuyen el lift a cada componente del sistema de forma inequívoca —algo imposible con datos puramente observacionales.

3.1.2 Catálogo sintético y taxonomía

El catálogo sintético se genera de forma declarativa a partir de una taxonomía jerárquica: categoría → subcategoría → marca → atributos (género objetivo, banda de edad, banda de precio, estilo, color, material). Las combinaciones de la taxonomía se muestrean para producir aproximadamente 2 000 productos con títulos y descripciones en español, generadas por plantillas parametrizadas con suficiente variedad para que tanto la búsqueda léxica (BM25) como los embeddings semánticos tengan señal real y no trivial.

A cada producto le corresponde un vector latente de factores (**factor_vector**), que codifica sus atributos en el espacio de ground-truth; este vector **no es el embedding Voyage** sino la representación canónica del producto construida antes de cualquier proceso de embedding. Los embeddings Voyage se calculan del texto y se persisten por separado. El **factor_vector** es la fuente de verdad que permite después evaluar si un método de embedding recupera el gusto latente del usuario.

El catálogo se genera de forma determinista dado un valor de semilla (**--seed**), garantizando la reproducibilidad de todos los experimentos. Los campos de negocio necesarios para F4 (**margin_pct**, **stock_health**, **seller_id**, **seller_age_days**) se añaden al catálogo en esa fase como extensión aditiva, sin alterar la estructura de datos de fases previas.

Diseño de referencia: docs/superpowers/specs/2026-05-29-thesis-f0-f1-data-eval-embeddings-design.md §4.2.

3.1.3 Grafo de complementariedad ground-truth

Sobre la taxonomía se define un grafo de complementariedad con reglas semánticas conocidas: un teléfono complementa a una funda, un cargador y unos auriculares; un vestido complementa a tacones, cartera y collar; dos productos de la misma subcategoría pero diferente marca son sustitutos; una variante de gama superior constituye un upgrade. Este grafo se persiste en la tabla `gt_product_relations` con los tipos `complement`, `substitute`, `upgrade` y `accessory`, así como un campo de fuerza de la relación.

La hipótesis central que articula los experimentos de F1 y F3 es que la complementariedad comercial no es recuperable por similitud de texto (dos productos pueden ser complementos perfectos sin compartir vocabulario), pero sí emerge de la co-ocurrencia de comportamiento cuando el generador de sesiones siembra co-visitas que siguen el grafo. Esta hipótesis es **verificable** porque el grafo es conocido, lo que convierte su recuperabilidad en un test de discriminación del arnés.

Diseño de referencia: docs/superpowers/specs/2026-05-29-thesis-f0-f1-data-eval-embeddings-design.md §4.3.

3.1.4 Generador de comportamiento con estado latente conocido

El generador de comportamiento produce sesiones de usuario a partir de un modelo generativo parametrizado. Cada usuario simulado (`sim_users`) posee un estado latente que incluye: una mezcla de $K \in [1, 3]$ clusters de gusto definidos en el espacio de factores, una sensibilidad al precio (`price_sensitivity`), una propensión a comprar para otros (`p_gift`), y entre cero y tres destinatarios (`sim_user_recipients`) con relación, género y banda de edad conocidos.

Al generar cada sesión, el modelo decide con probabilidad `p_gift` si se trata de una sesión de regalo o de compra propia; registra la intención real (`intent`) en `sim_sessions` junto con el identificador del destinatario cuando corresponde. El modelo de click y compra sigue una cadena `view` $\rightarrow$ `cart` $\rightarrow$ `purchase` con probabilidades de embudo configurables; la probabilidad de ver un ítem es proporcional a su similitud con la intención latente de la sesión, penalizada por la desviación del precio respecto al presupuesto del usuario y bonificada por la log-popularidad. La co-ocurrencia intra-sesión se **siembra** desde el grafo de complementariedad ground-truth: los complementos del ítem de anclaje aparecen con mayor frecuencia en la misma sesión, generando la señal de co-compra que el algoritmo NPMP explora en F3.

El generador es completamente determinista dado su valor de semilla. La “próxima compra” reservada para evaluación se persiste en la tabla `holdout`.

Diseño de referencia: docs/superpowers/specs/2026-05-29-thesis-f0-f1-data-eval-embeddings-design.md §4.4.

3.1.5 Esquema de base de datos dedicado

Todos los datos del programa se alojan en el esquema Postgres `thesis`, paralelo al esquema de producción (`public`) y al de pruebas (`test_schema`). El esquema replica las tablas que el pipeline de producción consume (`products`, `events`, `user_profiles`, `session_vectors`, `co_occurrence`, `co_occurrence_top`, `cohort_centroids`, etc.) y añade las tablas de ground-truth: `gt_product_factors`, `gt_product_relations`, `sim_users`, `sim_user_recipients`, `sim_sessions` y `holdout`. Este aislamiento garantiza que los experimentos de tesis no afectan los datos de producción en ningún caso.

3.2 Por qué el ground-truth es imprescindible

La ventaja fundamental del simulador sobre un dataset observacional es que permite **preguntas contrafácticas** con respuesta conocida. Conocer el gusto latente del usuario (los clusters del estado latente) permite evaluar si el sistema recupera ítems del gusto correcto, no solo ítems populares. Conocer la intención por sesión (propia vs. regalo) permite segmentar el holdout sin etiquetar manualmente. Conocer el grafo de complementos permite medir si el sistema entiende la relación comercial entre productos y no solo la lingüística. Y conocer la próxima compra efectuada permite construir un holdout temporal estricto con **sin fuga de información** hacia el pasado.

Esta arquitectura experimental sigue el espíritu de los bancos de pruebas sintéticos usados en sistemas de recomendación de investigación (world models, AlignUSER), pero está diseñada en función de las preguntas comerciales concretas del negocio: recuperación del gusto personal, cross-sell de complementos, personalización de regalos y optimización multi-objetivo del feed.

El holdout temporal se construye con un split leave-one-out por usuario: la última sesión de compra pasa a test; el resto permanece en train. Esto simula la condición de producción (predecir la siguiente compra dado el historial pasado) y previene el look-ahead bias que afecta a los splits aleatorios.

3.3 El arnés de evaluación

3.3.1 Arquitectura de interfaces

El arnés de evaluación se articula alrededor de dos interfaces abstractas. La interfaz **Ranker** recibe un contexto de usuario y un conjunto candidato y devuelve una lista de identificadores de producto ordenados. La interfaz **Embedder** produce vectores a partir de productos o consultas de usuario y computa scores. Todos los métodos estudiados en el programa —baselines, estrategias de embedding de F1, representación multi-vector de F2, rerankers de F3 y el scorer multi-objetivo de F4— implementan la interfaz **Ranker**, lo que garantiza la comparabilidad uniforme en todas las ablations.

3.3.2 Suite de métricas

La suite de métricas cubre tres dimensiones:

Métricas de accuracy. Recall@k mide la fracción de usuarios para los que el ítem holdout aparece en los primeros k resultados. nDCG@k (Normalized Discounted Cumulative Gain) pondera la posición del ítem relevante de forma logarítmica. MRR (Mean Reciprocal Rank) captura la posición media del primer ítem relevante. MAP (Mean Average Precision) integra la precisión a lo largo del ranking completo. HitRate@k es un indicador binario de presencia en el top-k.

Métricas beyond-accuracy. La diversidad intra-lista se mide como $1 - \overline{\text{coseno por pares}}$ entre los ítems del top-k. La novedad se mide como $-\log$ de la popularidad del ítem. El Gini de exposición de vendedores cuantifica la concentración del tráfico entre los vendedores del catálogo: un Gini cercano a cero indica exposición equitativa, mientras que un Gini elevado señala hiperconcentración.

Métricas comerciales y de tarea. complement-recall@k mide la fracción de complementos ground-truth del último ítem visto que aparecen en los primeros k resultados —es el indicador central de cross-sell recuperado por el sistema—. recipient-fit@k, introducida en F2, mide en

sesiones de regalo la fracción del top-k cuyo género objetivo y banda de edad caen dentro del perfil del destinatario real registrado en `sim_user_recipients.set-change@10`, introducida en F3, cuantifica la fracción del top-10 de un reranker que difiere del top-10 del orden de base del pool, distinguiendo el reordenamiento puro de la verdadera selección. `revenue@k`, introducida en F4, estima el ingreso esperado del feed como la suma del revenue esperado de los k primeros ítems, siendo el revenue de cada ítem el producto de la probabilidad de compra, el precio y el margen.

3.3.3 Baselines

Cuatro baselines establecen los pisos de referencia: (a) random, que ordena el catálogo sin señal; (b) popular-global, que ordena por popularidad absoluta sin personalización; (c) popular-cohort, que ordena por popularidad dentro del cohort demográfico del usuario; y (d) cosine-single-vector, que corresponde al sistema actual de producción —un vector de perfil único por usuario comparado por coseno con los embeddings de texto Voyage. La progresión de baselines permite aislar la contribución de cada componente.

3.3.4 Estimadores off-policy (OPE)

Para conectar las métricas offline con el impacto esperado en producción, el arnés implementa tres estimadores de evaluación off-policy (OPE): IPS (Inverse Propensity Scoring), SNIPS (Self-Normalized IPS) y el estimador doblemente robusto (DR). La ventaja del entorno sintético es que las propensidades (probabilidades de que el sistema logged presentara cada ítem) son conocidas por diseño —son parámetros del generador—, lo que permite validar los estimadores OPE contra la verdad y medir su varianza sin las aproximaciones que exigen los logs de producción reales.

3.3.5 Ablation runner y reproducibilidad

El ablation runner recorre el producto cartesiano de configuraciones (embedder $\times$ baseline $\times$ parámetros) y agrega las métricas en tablas Markdown y JSON listos para incluir en los capítulos de resultados. Todo el arnés es determinista bajo una semilla fija: la misma semilla produce los mismos datos, los mismos splits, las mismas métricas, garantizando la reproducibilidad plena de todos los experimentos de la tesis.

3.4 Disciplina de datos reales: sin mocks

Una restricción de diseño central del programa es que **ningún estudio se ejecuta contra mocks de base de datos, de embeddings ni de LLMs**. Todos los experimentos corren contra Postgres real (con la extensión `pgvector`) y contra la API Voyage real para los embeddings. Los estudios de F3 que involucran reranking con LLM utilizan DeepSeek a través del proveedor configurado (`defaultProvider`), sin interceptación simulada.

Esta restricción se **hace cumplir automáticamente** mediante un verificador AST integrado en el repositorio que detecta cualquier patrón de mock en el código de los estudios (`pnpm test:quality`). El objetivo no es solo el rigor académico: en producción, cada llamada al agregador de precios tiene costo real y latencia real. Un sistema evaluado íntegramente sobre mocks puede mostrar métricas excelentes que se degradan significativamente en producción porque las llamadas reales tienen distribuciones de latencia, fallos parciales y costos que los mocks no reproducen. La evaluación debe reflejar el sistema real para que las conclusiones sean transferibles.

Este principio se extiende también a los costos: los embeddings Voyage tienen un precio por token, y el arnés los calcula una sola vez y los persiste en la base de datos por hash de texto (caché), de modo que los re-runs de experimentos no incurren en costos adicionales. El LLM reranker de F3 opera sobre ventanas de 30 candidatos por consulta para mantener el costo controlado, y el runner registra la tasa de fallback del LLM (el porcentaje de veces que el modelo no devuelve un resultado válido y se activa el orden determinista de respaldo); el fallback rate es una métrica reportada explícitamente para la familia LLM.

3.5 Diseño de las fases experimentales

El programa se organiza en cinco fases. Las cuatro primeras (F0–F4) son experimentales y corresponden a contribuciones empíricas diferenciadas; F5 corresponde a la escritura y el piloto de producción. Cada fase construye sobre la anterior: reutiliza el mismo dataset, el mismo arnés y el mismo esquema de base de datos, extendiendo el aparato en lugar de reemplazarlo.

3.5.1 F0: Fundación de datos y arnés de evaluación

F0 establece los cimientos: genera el catálogo sintético, el grafo de complementariedad y el comportamiento de usuarios; construye el arnés de evaluación con su suite de métricas y baselines; y ejecuta una primera corrida de validación del arnés (test de discriminación: el grafo ground-truth es recuperable por co-ocurrencia pero no por coseno de texto). F0 no reporta resultados de ningún método de personalización; su criterio de aceptación es que el arnés distingue correctamente el método que debería ganar.

Diseño de referencia: docs/superpowers/specs/2026-05-29-thesis-f0-f1-data-eval-embeddings-design.md

3.5.2 F1: Estudio comparativo de embeddings

F1 estudia seis estrategias de embedding bajo una interfaz común **Embedder**: E0 (texto, Voyage, línea base actual), E1 (Prod2Vec, skip-gram sobre secuencias de sesión), E2 (híbrido con gate adaptativo según número de interacciones), E3 (two-tower con corrección logQ del sesgo de muestreo, según Yi et al., RecSys’19), E4 (late-interaction MaxSim aproximado sobre chunks sin GPU) y E5 (voyage-context-3, candidato realista de serving). La comparación se realiza sobre tres preguntas comerciales que el coseno de texto no resuelve bien: recuperación del gusto personal (próxima compra), recuperación de complementos (cross-sell) y recuperación de cola larga. Todas las estrategias se evalúan con el arnés F0 sobre el mismo holdout.

Diseño de referencia: docs/superpowers/specs/2026-05-29-thesis-f0-f1-data-eval-embeddings-design.md §4.7.

3.5.3 F2: Representación multi-vector del usuario y modelo de regalo

F2 ataca dos limitaciones del vector único que F1 puso de manifiesto: el usuario de intereses múltiples y ortogonales queda representado por un vector promedio que no refleja ninguno de sus gustos, y el usuario que compra regalos contamina su perfil con las preferencias ajenas. F2 implementa la representación multi-vector PinnerSage (Pal et al., KDD’20): clustering aglomerativo de enlace promedio con distancia coseno sobre el historial en el espacio E1, con un número de modos adaptativo por usuario y cada modo representado por su medoides (ítem real, interpretable). La detección de intención de regalo se realiza a nivel de sesión mediante señales de desviación cross-cohort (la sesión

apunta a una demografía alejada del perfil propio) sin acceder al ground-truth de intención en inferencia. El vector de destinatario es efímero: se construye a partir de los ítems de la sesión de regalo y no contamina los modos permanentes del usuario. El retrieval multi-modo asigna cuotas proporcionales al peso de cada modo y fusiona las listas con RRF. La evaluación es segmentada por intención real (self/gift) y nivel de multimodalidad del usuario.

Diseño de referencia: docs/superpowers/specs/2026-05-29-thesis-f2-multivector-recipient-gift-design

3.5.4 F3: Pool de candidatos multi-fuente y comparación de rerankers

F3 parte de un hallazgo del audit conductual previo al programa: el reranker LLM del sistema anterior “no cambiaba el conjunto, solo reordenaba”. La hipótesis de F3 es que el problema era el pool de entrada, no el reranker. Con un pool grande (100–300 candidatos) generado por cuatro fuentes complementarias —retrieval F2 multi-modo, vecinos NPMI de co-compra del último ítem visto, popularidad por cohort/destinatario, y cuota de exploración/novedad—, un reranker con acceso a features que el retrieval no ve (score de co-compra NPMI, ajuste de precio, señal de regalo de F2, recencia, fuente del candidato) sí puede seleccionar y reordenar con lift medible. F3 compara cuatro familias de reranker sobre el mismo pool: LTR aprendido (regresión logística con SGD, entrenada solo sobre el split train), LLM listwise (DeepSeek, ventana de 30 candidatos), cross-encoder MaxSim (aproximación late-interaction sin GPU, reutilizando E4 de F1) y baselines (MMR y orden RRF del pool). La métrica `set-change@10` evalúa directamente si el reranker cambia el conjunto recuperado o solo lo reordena, distinguiendo cambio de ruido.

El grafo NPMI se construye sobre los eventos del simulador mediante backfill en el esquema `thesis`, con la misma función `recomputeNPMI` usada en producción pero operando sobre el search path `thesis`. Los vecinos NPMI de cada producto deben recuperar los complementos del grafo ground-truth de F0 mucho mejor que el coseno, lo que se verifica con el mismo test de discriminación del arnés.

Diseño de referencia: docs/superpowers/specs/2026-06-07-thesis-f3-candidate-generation-rerank-study

3.5.5 F4: Ranking multi-objetivo

F4 cierra el arco del programa. El resultado honesto de F3 es que los rerankers con features no superan al RRF del pool en relevancia pura porque el ranking estaba optimizando un solo objetivo. F4 demuestra que el reranking sí aporta cuando se negocian objetivos competidores: $s(p|u) = \sum_k \lambda_k \cdot f_k(p, u)$. Las features objetivo son: relevancia (coseno a los modos F2 del usuario), margen del producto, probabilidad de conversión (derivada del mismo modelo de afinidad latente del generador, para consistencia), novedad, diversidad marginal (estilo MMR greedy) y fairness de vendedores (boost a vendedores con poca exposición).

Un elemento de diseño clave para que exista una negociación real es la **anti-correlación intencional** entre margen y popularidad/precio en el catálogo: los productos más relevantes y visibles tienden a tener menor margen, mientras que la cola larga tiende a mayor margen. Sin esta anti-correlación, el ítem más relevante sería también el más rentable y no habría trade-off que demostrar. La anti-correlación está sembrada en el generador y es verificable.

El modelo de outcome `expectedRevenue = P(\text{compra}|u,p) \cdot \text{precio} \cdot \text{margen}\$` se usa tanto como feature estimable en inferencia (`convProb`) como para medir el revenue del feed (`revenue@k'`); la probabilidad de compra estimable y la compra holdout son señales distintas, lo que evita el leakage del split de test.

El sweep de la grilla de λ es determinista y produce un vector de métricas para cada configuración; de esas configuraciones se extrae la **frontera de Pareto** (el conjunto no-dominado) y se selecciona el **punto KPI**: la configuración que maximiza `revenue@10` sujeta a guardrails de relevancia mínima y fairness mínima. Este punto es la respuesta del programa a la pregunta del negocio: cómo convertir el ranking de una decisión implícita (relevancia pura) en una decisión de negocio explícita y medible.

Diseño de referencia: `docs/superpowers/specs/2026-06-07-thesis-f4-multi-objective-ranking-design.md`

3.6 Advertencia de escala y validez externa

3.6.1 Escalas de datos y separación de tablas de resultados

El programa opera con dos escalas de dataset que no son comparables directamente. La evaluación inicial del arnés (F0) se ejecutó sobre un catálogo de aproximadamente $n = 400$ productos, suficiente para validar la corrección del arnés pero insuficiente para resultados representativos. Los estudios comparativos de F1 a F4 corren sobre el catálogo regenerado a $n = 2000$ productos, con el grafo de complementariedad completo y los campos de negocio de F4. Las cifras de ambas escalas **no se mezclan en ninguna tabla de resultados**: cada tabla indica explícitamente la escala del dataset sobre la que fue producida.

3.6.2 Validez externa: dataset sintético y cross-check público

El dataset es enteramente sintético. Las conclusiones sobre qué métodos son superiores se obtienen bajo las condiciones de un marketplace simulado, con distribuciones de comportamiento y complementariedad generadas por reglas conocidas. Esta es la fortaleza metodológica que permite los ablations rigurosos, pero es también una limitación de validez externa.

Para mitigar esta limitación, el diseño de F0 contempla un adaptador de dataset público (`scripts/thesis/data/public-adapter.ts`) que carga un dataset abierto de e-commerce y lo mapea al esquema `thesis`, permitiendo un cross-check de las conclusiones principales sobre comportamiento real. Este adaptador está implementado en el repositorio con la anotación `scope:'thesis'` y el marcador `source='public'`, y está listo para recibir un dataset real (candidatos: Amazon Reviews 2023 en categorías de moda/electrónica, o datasets de sesiones tipo RetailRocket/Yoochoose según licencia). La ejecución de ese cross-check queda como trabajo futuro inmediato: el único bloqueante es la elección y carga del dataset real, no el código del adaptador ni del arnés.

Toda afirmación sobre ventajas de un método sobre otro en este trabajo debe leerse con esta acotación: los resultados son válidos internamente bajo las condiciones del simulador, y la validez externa está pendiente de confirmación empírica sobre datos reales.

3.7 Reproducibilidad y separación de código

Todo el código experimental del programa reside bajo `src/thesis/` (librería pura) y `scripts/thesis/` (CLIs de generación, entrenamiento y ejecución de estudios), aislado del código de producción (`src/sectors/d-personalization/`). Los experimentos no modifican las tablas del esquema de producción bajo ninguna circunstancia. Los estudios importan funciones de producción cuando corresponde (por ejemplo, `rrfFuse`, `mmrSelect`, `recomputeNPMI`), pero las

invocan a través del cliente de base de datos con `scope:"thesis"` para que operen sobre el search path del esquema experimental.

La reproducibilidad está garantizada por tres mecanismos: (1) toda generación de datos y toda corrida de evaluación acepta un parámetro de semilla explícito que determina completamente el resultado; (2) los embeddings calculados con la API Voyage se persisten por hash de texto en la base de datos, de modo que re-runs no incurrir en variabilidad ni en costo adicional; y (3) el verificador AST integrado rechaza cualquier patrón de mock, asegurando que los artefactos presentados en la tesis son reproducibles desde el mismo entorno real.

4 Embeddings comerciales (F1)

4.1 Motivación y pregunta de investigación

El primer factor del programa de tesis (F1) se ocupa de la capa más fundamental del pipeline de personalización: la representación vectorial del producto. Antes de aplicar cualquier reranker o función de ranking multi-objetivo, el sistema debe recuperar un conjunto de candidatos prometedores. La calidad de ese conjunto determina el techo práctico de todo lo que viene después.

La pregunta central es triple: ¿qué representación de producto recupera mejor (a) la próxima compra probable del usuario, (b) los complementos funcionales de lo que ya adquirió, y (c) la cola larga del catálogo, aquellos ítems que satisfacen necesidades latentes poco frecuentes? La hipótesis de partida era que los embeddings comportamentales —entrenados directamente sobre co-ocurrencias de compra— superarían a los embeddings de texto en (a) y (b), mientras que los modelos contextualizados podrían aportar en (c). Esta hipótesis se confirmó parcialmente, pero reveló un hallazgo negativo de mayor relevancia para la arquitectura final del sistema.

4.2 Diseño experimental: comparación justa entre espacios vectoriales

Se evaluaron seis estrategias de representación, todas expuestas a través de una interfaz común de recuperación vectorial (TopK sobre pgvector):

- **e0_text**: embedding de texto puro con Voyage AI (1024 dimensiones), concatenando título, categoría y descripción corta del producto.
- **e1_prod2vec**: embedding comportamental entrenado con Prod2Vec sobre el historial de compras (64 dimensiones), análogo al Item2Vec de la literatura de sistemas de recomendación.
- **e2_hybrid**: fusión de score entre el espacio de texto (1024-d) y el comportamental (64-d), sin reentrenamiento conjunto.
- **e3_two_tower**: two-tower entrenado con muestras in-batch y corrección logQ para descontar la popularidad como sesgo de muestreo (64 dimensiones).
- **e4_late**: interacción tardía (late-interaction) con representación chunk-MaxSim; la consulta del usuario se descompone en hasta 24 fragmentos y se agrega por máximo de similitud.
- **e5_context3**: Voyage `context-3`, un modelo de embebido contextualizado que incorpora información de sesión (1024 dimensiones).

4.2.1 Garantías de equidad (correcciones P0)

La comparación justa entre espacios vectoriales exige que la ventaja observada refleje la calidad de la representación y no artefactos del protocolo. Tres problemas de equidad se detectaron y corrigieron mediante revisión adversarial antes de ejecutar el experimento definitivo:

1. **Universo común de candidatos.** Cada espacio vectorial sólo puede representar los productos que ingirió durante su entrenamiento o indexación. Comparar sobre conjuntos de candidatos diferentes introduce sesgo estructural. La solución fue restringir el universo a los **1999 ítems** representables en todos los espacios simultáneamente.
2. **Intersección de complementos.** Los objetivos de `complement-recall@10` se restringieron a productos que pertenecen al mismo universo común y no forman parte del historial de entrenamiento del usuario, asegurando que la métrica mida recuperación real y no cobertura diferencial de los espacios.
3. **Híbrido seguro en dimensión.** La fusión de score en `e2_hybrid` opera sobre las puntuaciones de similitud normalizadas, no sobre concatenación de vectores, evitando que la mayor dimensionalidad del espacio de texto domine artificialmente la similitud resultante.

Con estas correcciones en vigor, los 1098 casos de evaluación y los candidatos son idénticos para los seis espacios; sólo varía el vector que determina el orden.

4.3 Resultados

La tabla siguiente recoge las métricas principales sobre el conjunto de evaluación completo ($n = 1098$ casos):

Espacio	nDCG@10	Recall@10	complement-recall@10
<code>e2_hybrid</code>	0.124	0.252	0.000
<code>e1_prod2vec</code>	0.101	0.219	0.000
<code>e3_two_tower</code>	0.049	0.094	0.000
<code>e4_late</code>	0.039	0.087	0.002
<code>e5_context3</code>	0.039	0.085	0.001
<code>e0_text</code>	0.038	0.086	0.001

4.4 Hallazgos

4.4.1 Los embeddings comportamentales superan al texto en relevancia

El resultado más nítido es la brecha entre los espacios comportamentales y el texto puro. `e2_hybrid` obtiene un `nDCG@10` de 0.124 y un `Recall@10` de 0.252, frente a 0.038 y 0.086 del embedding de texto `e0_text`. La ratio es aproximadamente $3.3\times$ en `nDCG@10` y $2.9\times$ en `Recall@10`. `e1_prod2vec`, sin ninguna componente textual, ya supera al texto en ambas métricas (0.101 vs. 0.038 en `nDCG@10`, una ratio de $\approx 2.7\times$; 0.219 vs. 0.086 en `Recall@10`).

Esta superioridad confirma la intuición de base: en un catálogo de reventa donde los usuarios compran ítems relacionados con su contexto de vida (no con las palabras del título), la geometría del espacio comportamental —aprendida directamente de co-compras— captura la relevancia percibida mejor que la semántica lexical.

El modelo `e3_two_tower`, pese a incorporar señal comportamental, queda considerablemente por debajo de `e1_prod2vec` (0.049 vs. 0.101 en `nDCG@10`). Una hipótesis es que el conjunto de entrenamiento disponible es insuficiente para que el two-tower generalice bien en un catálogo de cola larga; Prod2Vec, al ser más simple, sufre menos de sobreajuste ante datos escasos.

Los modelos de interacción tardía y contextualizado (`e4_late`, `e5_context3`) se agrupan junto con `e0_text` en la banda baja (0.038–0.039 en `nDCG@10`), sin ventaja apreciable sobre el texto puro bajo las condiciones de este experimento.

4.4.2 Hallazgo negativo: los embeddings no recuperan complementos

El resultado más importante de F1 —y el menos esperado— es que la columna `complement-recall@10` es prácticamente cero para los seis espacios: los valores oscilan entre 0.000 y 0.002. Dicho de otro modo, en un universo de 1999 candidatos rankeados por similitud vectorial al perfil del usuario, los complementos funcionales del producto adquirido no aparecen en los primeros diez puestos de ningún espacio, ni siquiera en los comportamentales.

Este hallazgo se reporta con honestidad plena porque constituye una contribución de diseño, no un fracaso a ocultar. Su implicación arquitectónica es directa: **el coseno en el espacio de embedding no es la herramienta adecuada para el cross-sell**. La complementariedad funcional —“quien compra X también necesita Y”— es una relación asimétrica y específica que los embeddings capturan sólo marginalmente; lo que sí capturan es similitud semántica o co-preferencia, que predice la próxima compra pero no el ítem complementario.

La conclusión de programa es que el cross-sell debe resolverse en el grafo de co-ocurrencia o mediante puntuaciones NPMI (Normalized Pointwise Mutual Information) sobre pares de productos, no mediante recuperación por coseno. Los embeddings son la herramienta correcta para relevancia y descubrimiento; el grafo de co-compra es la herramienta correcta para complementariedad. Ambas fuentes deben coexistir en el pool de candidatos de la etapa de fusión (F3), cada una aportando una señal ortogonal a la otra.

4.4.3 Recomendación de producción

El análisis de utilidad normaliza la calidad frente al costo de indexación y consulta. Con la función $\text{utility} = \text{quality} - 0.5 \cdot \text{normalizedCost}$, `e2_hybrid` obtiene el mayor valor de calidad (0.074) y la mayor utilidad relativa entre las opciones evaluadas.

La recomendación de producción es desplegar `e2_hybrid`: un vector denso construido por fusión de score entre el espacio de texto Voyage y el espacio Prod2Vec, sin reentrenamiento conjunto. La ventaja operativa es que es un vector denso compatible con pgvector sin modificaciones de esquema, la inferencia en línea combina dos llamadas ligeras, y la calidad de recuperación es 3.3× superior al embedding de texto puro que tendría el sistema sin señal comportamental.

4.5 Síntesis del capítulo

F1 establece que la representación vectorial comportamental es imprescindible para la calidad de recuperación en este dominio. La fusión híbrida texto-comportamiento (`e2_hybrid`) ofrece el mejor balance calidad-costo y es la base sobre la que se construye el pool de candidatos en etapas posteriores del programa. El hallazgo negativo sobre complementariedad delimita el alcance arquitectónico de los embeddings y motiva el diseño del componente de cross-sell basado en grafo, que se desarrolla en la etapa de fusión de candidatos (F3).

5 Usuario multi-vector y modelo de regalo (F2)

5.1 Motivación y pregunta de investigación

El segundo factor del programa de tesis (F2) parte de una crítica conceptual al modelo de usuario heredado de F1: representar al comprador como un único vector de perfil equivale a afirmar que sus intereses son unimodales y estables. Esta asunción falla en al menos dos regímenes frecuentes en e-commerce: el usuario que alterna entre categorías heterogéneas según el momento de vida (e.g., artículos de trabajo y artículos de ocio en la misma semana), y el usuario que compra para un tercero —una situación que denominamos *sesión de regalo* y que contamina el perfil propio si se registra de la misma forma que una compra de autoconsumo.

La pregunta de investigación es doble. Primera: ¿produce un sistema de recuperación multi-vector —donde el perfil del usuario es una *distribución* de intereses en lugar de un punto único— mejores listas de candidatos que el vector único de F1, y el beneficio es consistente en todos los segmentos de usuario? Segunda: ¿puede el sistema reconocer automáticamente una sesión de regalo y construir, para esa sesión y sólo para ella, un vector efímero de destinatario que apunte a los intereses del receptor sin alterar el perfil del comprador?

5.2 Diseño: un usuario como distribución de intereses

5.2.1 Representación multi-vector

La inspiración directa es PinnerSage [pal2020pinnerSage], el sistema de Pinterest que modela cada usuario como una mezcla de medoides, cada uno representando un clúster de intereses distinto. La implementación de este trabajo sigue el mismo espíritu con tres decisiones de diseño:

1. **Clustering aglomerativo coseno.** A partir del historial de interacciones del usuario expresado en el espacio `e1_prod2vec` (64 dimensiones), se aplica clustering aglomerativo de enlace promedio con distancia coseno. El número de modos emerge de los datos: un usuario con historial estrecho produce un único clúster; un usuario con intereses dispersos puede producir dos o tres.
2. **Medoides como representantes.** Cada clúster queda representado por su medoide —el ítem real más cercano al centroide empírico—, lo que garantiza que el vector de consulta pertenece al espacio de productos reales y no a un punto interpolado sin referente semántico.
3. **Invarianza al orden.** La asignación de clústeres es independiente del orden cronológico de las interacciones, característica heredada del diseño de PinnerSage y que hace el sistema robusto frente a diferentes secuencias de observación.

Durante la recuperación, cada modo emite una cuota proporcional de candidatos y los resultados se fusionan mediante *Reciprocal Rank Fusion* (RRF), asegurando que ningún interés minoritario quede excluido por la dominancia numérica del modo principal.

5.2.2 Modelo de regalo: vector efímero de destinatario

Una sesión de regalo presenta una distribución de compra radicalmente diferente a la compra de autoconsumo: los ítems observados reflejan el gusto del *destinatario*, no del comprador. Si ese historial se incorpora directamente al perfil permanente del comprador, introduce ruido que degrada las recomendaciones futuras de autoconsumo.

La solución adoptada es la detección heurística de sesiones de regalo seguida de la construcción de un **vector de destinatario efímero**: un vector de perfil calculado únicamente a partir de los ítems de la sesión corriente, utilizado sólo durante esa sesión y desechado al concluirla. El perfil permanente del comprador no se modifica.

La detección utiliza coherencia demográfica entre los ítems observados en la sesión y la cohorte del comprador, cruzando señales de género y edad. Cuando la sesión resulta incoherente con el perfil demográfico del comprador —por ejemplo, un hombre adulto añadiendo artículos de bebé de género femenino—, el sistema clasifica la sesión como regalo y activa el vector efímero.

5.3 Resultados

5.3.1 nDCG@10 segmentado: F1 vs F2

El espacio de ítems utilizado es `e1_prod2vec`, con universo común de 1999 productos y 1098 casos de evaluación —los mismos que en F1, para garantizar comparabilidad directa. La tabla siguiente reproduce los resultados exactos del estudio:

Segmento	n	F1-single	F2-multivec
overall	1098	0.101	0.152
self\ 1mode	598	0.151	0.200
self\ 2-3modes	151	0.109	0.163
gift\ 1mode	287	0.013	0.063
gift\ 2-3modes	62	0.006	0.072

F2 supera a F1-single en todos los segmentos sin excepción. La ganancia global es de 0.101 a 0.152, una mejora relativa del 51%. En los segmentos de autoconsumo, donde el vector único ya ofrecía rendimiento razonable (`self|1mode` 0.151), F2 lo eleva a 0.200, confirmando que la multi-modalidad aporta incluso cuando el usuario es relativamente unifocal.

El efecto más pronunciado aparece en las sesiones de regalo. El vector único **colapsa** en `gift|2-3modes` hasta un nDCG@10 de 0.006 —prácticamente aleatorio—, mientras que F2 alcanza 0.072, una mejora de aproximadamente 12×. En `gift|1mode` la ganancia es de 0.013 a 0.063 ($\approx 5\times$). Este contraste no es sorprendente: el vector único del comprador, calibrado sobre intereses de autoconsumo, señala en dirección opuesta a lo que el destinatario desea.

5.3.2 Recipient-fit@10: ¿apunta el feed al destinatario?

Para las sesiones clasificadas como regalo ($n = 349$), se calculó una métrica adicional denominada **recipient-fit@10**: la fracción de los 10 candidatos recuperados que pertenecen a las categorías demográficas del destinatario inferido. Esta métrica captura si el feed resultante *apunta al receptor correcto*, independientemente de si los ítems exactos son los adquiridos.

Los resultados son:

- **F1-single:** `recipient-fit@10` = 0.285
- **F2-multivec:** `recipient-fit@10` = 0.476

El feed de regalo de F2 apunta al destinatario aproximadamente 1.7 veces mejor que el vector único. Expresado en términos absolutos, casi la mitad de los candidatos en el top-10 son relevantes para el receptor ($\approx 47.6\%$), frente a poco más de una cuarta parte con F1-single (28.5%).

5.3.3 Calidad de la detección de regalo

La detección heurística de sesiones de regalo es, como cualquier heurística, imperfecta. La evaluación contra la verdad de fondo disponible ($n = 1098$ sesiones totales) arroja los siguientes resultados:

Métrica	Valor
Precisión	0.430
Recall	0.387
F1	0.407

La matriz de confusión desagregada muestra: $TP = 135$, $FP = 179$, $FN = 214$, $TN = 570$.

Estas cifras deben leerse con honestidad: el detector comete aproximadamente un error por cada 1.7 detecciones positivas ($FP = 179$ frente a $TP = 135$), y deja escapar el 61% de las sesiones de regalo verdaderas ($FN = 214$ de 349). La consecuencia práctica es que F2 activa el vector efímero en muchas sesiones que no son de regalo —introduciendo ruido en lugar de señal—, y no lo activa en la mayoría de las que sí lo son.

Este es el **límite más importante del diseño F2**: la ganancia medida sobre `nDCG@10` y `recipient-fit@10` combina el efecto de la representación multi-vector (beneficio real) con los errores del detector (atenuación). El beneficio real de la multi-vectorialidad es posiblemente mayor de lo que los números indican; el modelo de regalo podría extraer más valor con un detector de mayor calidad —por ejemplo, con señales explícitas del usuario o inferencia demográfica supervisada.

En el estado actual, la precisión de detección de ≈ 0.43 se reporta explícitamente como restricción del sistema, no se oculta. Los resultados de F2 son válidos como medida del pipeline completo, incluyendo sus errores de clasificación.

5.4 Discusión y hallazgos

Dos tesis quedan empíricamente respaldadas por este estudio:

Tesis 1 — El usuario es una distribución, no un punto. La representación multi-vector produce listas de candidatos superiores en todos los segmentos evaluados. La multi-modalidad aporta incluso a usuarios unimodales (`self|1mode`: +32% relativo), y es especialmente crítica cuando los intereses del usuario son dispersos o cuando la sesión corriente refleja intereses ajenos. El paradigma del vector único es un caso degenerado del modelo multi-vector que sólo es óptimo bajo el supuesto —raramente satisfecho— de que todos los intereses del usuario son representables por un único punto del espacio vectorial.

Tesis 2 — El regalo requiere representación propia. Tratar una sesión de regalo como compra de autoconsumo destruye la calidad de recuperación: `nDCG@10` cae a valores cercanos a cero (0.006 en `gift|2-3modes` con F1-single). El vector efímero de destinatario, aun construido con un detector imperfecto, multiplica por ≈ 12 la métrica de relevancia en el peor segmento y eleva el `recipient-fit@10` de 0.285 a 0.476. La separación arquitectónica entre perfil del comprador y señal del destinatario es, por tanto, una decisión de diseño justificada empíricamente, no una complejidad gratuita.

La ganancia global de F2 sobre F1 —de `nDCG@10` 0.101 a 0.152— establece el nuevo techo del retriever sobre el que operarán el reranker (F3) y el ranking multi-objetivo (F4).

6 Generación de candidatos y reranking (F3)

6.1 Motivación y diagnóstico previo

Al concluir F2, el arnés de evaluación adversarial detectó un síntoma perturbador: el reranker LLM que se había incorporado al pipeline no cambiaba el conjunto de ítems recuperados. Los diez candidatos que devolvía eran, prácticamente, los mismos diez que producía la fusión MMR —reordenados pero no renovados. La hipótesis de trabajo fue que el reranker *estaba hambriento*: operaba sobre un pool demasiado pequeño (los treinta candidatos que F2 emitía como top-30 por similitud coseno a los medoides) y, con tan poco material de entrada, carecía de señal diferencial respecto al retriever. Si el ítem correcto no estaba en el pool, ningún reranker podía elevarlo; si el pool era casi idéntico al top-10 final, el reranker sólo podía reordenar, no recuperar.

Esta observación convierte F3 en una investigación de dos preguntas independientes pero relacionadas. Primera: ¿puede un pool de candidatos multi-fuente, sustancialmente más grande y más diverso, capturar más ítems relevantes que el top-30 de similitud coseno? Segunda: sobre ese pool ampliado, ¿qué familia de reranker —aprendido, probabilístico, neuronal profundo o por modelo de lenguaje— extrae mejor señal de relevancia?

6.2 Diseño del sistema F3

6.2.1 Pool de candidatos multi-fuente

El pool de F3 tiene tamaño 200 — $6.7\times$ más grande que el top-30 de F2— y se construye fusionando cuatro fuentes de candidatos mediante *Reciprocal Rank Fusion* (RRF):

1. **Retrieval por similitud coseno.** Los 80 ítems de mayor similitud coseno a los medoides del usuario (espacio `e1_prod2vec`), el mismo mecanismo que F2 utilizaba como única fuente.
2. **Co-ocurrencia NPML.** Los vecinos de compra conjunta del último ítem observado, puntuados por *Normalized Pointwise Mutual Information*. Esta fuente captura relaciones de complementariedad que la similitud coseno de texto no puede recuperar: un ratón inalámbrico y su alfombrilla pueden estar en categorías léxicamente distantes pero comprarse juntos de forma consistente.
3. **Popularidad por cohorte.** Ítems populares dentro de la cohorte demográfica del usuario (perfil de edad y género inferido). Esta fuente actúa como red de seguridad: cuando el historial del usuario es corto o ruidoso, los ítems populares en su cohorte son candidatos razonablemente seguros.
4. **Exploración aleatoria sembrada.** Un muestreo diversificado del catálogo con semilla determinista, cuya función es inyectar variedad y evitar que el pool quede enteramente dominado por las categorías más frecuentes en el historial del usuario.

Las cuatro listas se fusionan vía RRF antes de truncar al tamaño 200. La fusión garantiza que ninguna fuente domina completamente la composición del pool: un ítem que aparece en el puesto 80 de similitud coseno y en el puesto 15 de popularidad por cohorte recibe una puntuación combinada superior a un ítem que lidera únicamente una de las fuentes.

El pool resultante es **compartido**: todos los rerankers evaluados en este experimento operan sobre el mismo conjunto de 200 candidatos por usuario, lo que hace que sus métricas sean directamente comparables entre sí.

6.2.2 Cuatro familias de reranker

Sobre el pool compartido se evaluaron cuatro rerankers, elegidos para cubrir el espectro metodológico relevante:

- **baseline-rrf**: el orden producido por la propia fusión RRF al construir el pool. Es el punto de referencia: si ningún reranker lo supera, el valor de F3 está íntegramente en el pool, no en el reranker.
- **LTR (*Learning to Rank*)**: un modelo lineal puntual entrenado con descenso de gradiente sobre cinco *features* por ítem: **retrievalScore** (puntuación RRF del pool), **npmiScore** (puntuación de co-ocurrencia), **priceFit** (ajuste al rango de precios habitual del usuario), **demoMatch** (concordancia demográfica del ítem con la cohorte) y **popularity** (popularidad en cohorte). El modelo es supervisado: se entrena sobre el split de entrenamiento del arnés y se evalúa sobre el holdout temporal.
- **Cross-encoder MaxSim**: un modelo de interacción profunda que codifica la consulta (fragmentos **e4** de 1024 dimensiones de los ítems del historial de entrenamiento del usuario) y cada candidato en el mismo espacio de representación tardía (*late-interaction*, patrón F1), calculando la similitud máxima por fragmento. Una versión preliminar del experimento sufría un error de espacio vectorial: las consultas se construían en el espacio **e1** (64 dimensiones) y se comparaban contra documentos en el espacio **e4** (1024 dimensiones); **cosineSim** truncaba silenciosamente y producía métricas sin sentido (**nDCG@10** 0.027, **set-change@10** 0.952). La revisión final detectó y corrigió este P0, alineando ambos extremos al espacio **e4**.
- **LLM *listwise* con DeepSeek**: el reranker LLM que el audit adversarial había observado como inerte en F2. Recibe los 30 primeros ítems del pool ordenados por RRF y los reordena mediante un prompt de lista. En este experimento opera sobre un subconjunto de 120 casos por razones de coste y latencia, y se reporta por separado de la tabla principal.

6.2.3 Corrección del generador de datos sintéticos

Durante la preparación de los datos de entrenamiento del LTR se detectó un defecto en el generador del simulador: la siembra de complementos en el historial del usuario —especificada en §4.4 del documento de diseño— no había sido implementada. Sin ella, los datos de entrenamiento no contenían pares *ítem visto* $\rightarrow$ *complemento comprado*, lo que privaba al **npmiScore** de señal supervisada. El fix consistió en implementar la siembra de complementos tal como describía la especificación, regenerar los datos y reentrenar el LTR. Los resultados que se reportan a continuación corresponden al experimento con el generador corregido.

6.3 Resultados

6.3.1 Recall del pool: la ganancia principal

El primer resultado es también el más importante del factor F3. La tabla siguiente compara el recall del pool de 200 ítems con el recall del top-30 de F2, sobre los mismos 1098 casos de evaluación y el mismo espacio de ítems **e1_prod2vec** (universo de 1999 productos):

Configuración	Recall	Casos cubiertos
Pool F3 (200 ítems, multi-fuente)	0.839	921 / 1098
F2 top-30 (similitud coseno a medoides)	0.410	450 / 1098

El pool multi-fuente captura aproximadamente $2\times$ más compras futuras que el retriever coseno de F2. En términos absolutos, el ítem que el usuario comprará está presente en el pool en 921 de los 1098 casos de evaluación, frente a sólo 450 con el top-30. Este incremento de recall —de 0.410 a 0.839— establece el límite superior de lo que cualquier reranker puede lograr: si el ítem correcto no está en el pool, no puede aparecer en el top-10.

La contribución diferencial de la fuente NPMI merece mención explícita. Un test de discriminación commiteado (`tests/thesis/f3-cooccurrence.test.ts`, que aserchia `npmiHits > cosHits`) comparó, sobre casos con verdad de fondo conocida, cuántos ítems de la verdad de fondo (complementos esperados) recuperaba cada fuente; en la ejecución observada `npmiHits = 7` y `cosHits = 0`. La similitud coseno no recuperó ningún complemento presente en la verdad de fondo; la co-ocurrencia NPMI recuperó siete. Este resultado confirma empíricamente la premisa de F1: la complementariedad de compra no es capturada por los embeddings de texto, y una fuente dedicada a la co-ocurrencia aporta candidatos cualitativamente distintos.

6.3.2 Comparación de rerankers sobre el pool compartido

La tabla siguiente recoge las métricas de los cuatro rerankers sobre el pool compartido de 200 candidatos:

Reranker	nDCG@10	Recall@10	MRR	set-change@10
baseline-rrf	0.177	0.336	0.145	0.000
ltr	0.121	0.250	0.100	0.578
mmr	0.125	0.204	0.119	0.527
cross-encoder	0.055	0.120	0.053	0.821

La columna `set-change@10` mide la fracción de usuarios para los que el top-10 final difiere del top-10 de `baseline-rrf`. Un valor de 0.000 para `baseline-rrf` es tautológico (es el propio baseline); un valor de 0.821 para `cross-encoder` indica que en más de cuatro de cada cinco usuarios el cross-encoder propone un conjunto de candidatos radicalmente diferente al de la fusión RRF.

Este dato refuta directamente la observación del audit adversarial: **los rerankers de F3 sí cambian el conjunto recuperado**. El rango de `set-change@10` va de 0.527 (`mmr`) a 0.821 (`cross-encoder`), demostrando que cada reranker explora activamente el espacio de 200 candidatos y no se limita a reordenar los mismos diez que lidera el pool.

Sin embargo, y éste es el **hallazgo negativo central de F3**: ningún reranker supera al `baseline-rrf` en `nDCG@10`. El orden de relevancia producido por la propia fusión RRF al construir el pool es el mejor ranker disponible en este experimento, con `nDCG@10 = 0.177` y `Recall@10 = 0.336`. `mmr` alcanza `nDCG@10 = 0.125` con `set-change@10 = 0.527`: diversifica el top-10 pero lo hace a costa de precisión posicional. `ltr` obtiene `nDCG@10 = 0.121` y `Recall@10 = 0.250` pese a ser el único modelo supervisado. `cross-encoder` registra `nDCG@10 = 0.055`, el peor resultado en relevancia aunque el mayor en diversificación.

6.3.3 Reranker LLM listwise (DeepSeek)

El reranker LLM evaluado sobre los primeros 120 casos —operando sobre los 30 ítems de mayor puntuación RRF del pool— obtuvo `nDCG@10 = 0.170`, `Recall@10 = 0.350` y `set-change@10 = 0.427`, con una tasa de *fallback* de 0.000 (ninguno de los 120 prompts requirió recurrir al orden

de entrada). Estos números son virtualmente equivalentes al **baseline-rrf** en relevancia (0.170 vs 0.177) y confirman que, sobre el pool enriquecido, el LLM sí cambia el conjunto —a diferencia de lo observado en el audit adversarial— aunque no consigue una mejora neta en **nDCG@10**.

La diferencia respecto al comportamiento inerte detectado en F2 es precisamente el pool: en F2, el reranker LLM operaba sobre 10–30 candidatos muy similares al retriever coseno; en F3 dispone de 200 candidatos procedentes de cuatro fuentes complementarias. La hipótesis del audit —que el reranker estaba hambriento— queda así confirmada: el problema no era el modelo, sino la dieta.

6.3.4 Interpretabilidad del LTR: pesos de features

El modelo LTR ofrece una lectura directa de qué señales utiliza para reordenar. Los pesos aprendidos son:

Feature	Peso
retrievalScore	6.8185
popularity	2.2423
priceFit	1.8114
npmiScore	0.2878
demoMatch	0.0445
isGift	0.0000
(sesgo)	−15.6126

retrievalScore domina con diferencia (6.82), lo que indica que el modelo aprendido termina delegando en la puntuación RRF del pool —exactamente lo que hace el **baseline-rrf**—. **popularity** y **priceFit** tienen pesos moderados; **npmiScore** y **demoMatch** contribuyen marginalmente; **isGift** recibe peso cero, posiblemente porque en datos sintéticos la señal de regalo no discrimina suficientemente al nivel de ítem individual.

6.3.5 Segmentación: self vs gift

La comparación de **ltr** y **baseline-rrf** por segmento de intención revela una asimetría ya observada en F2:

Segmento	<i>n</i>	Reranker	nDCG@10	Recall@10	MRR
self	743	baseline-rrf	0.235	0.445	0.189
self	743	ltr	0.159	0.332	0.127
gift	355	baseline-rrf	0.055	0.107	0.055
gift	355	ltr	0.041	0.079	0.044

En sesiones de autoconsumo (**self**), **baseline-rrf** alcanza **nDCG@10** = 0.235 —el mejor valor registrado en todo el pipeline hasta F3. En sesiones de regalo (**gift**), la calidad cae drásticamente (0.055), lo que refleja que el pool de F3 hereda el problema de F2: la representación del destinatario sigue siendo débil cuando la detección de regalo falla. **ltr** subestima a **baseline-rrf** en ambos segmentos, confirmando que el patrón de dominancia del baseline no es un artefacto del promedio.

6.4 Discusión y hallazgos

Los resultados de F3 producen tres conclusiones bien delimitadas.

El valor de F3 reside en el pool, no en el reranker. El incremento de recall de 0.410 a 0.839 —la ganancia de $\approx 2\times$ — es el avance sustancial de este factor. Todos los rerankers evaluados operan *dentro* de ese espacio ampliado; ninguno lo supera en relevancia posicional. Si el objetivo es maximizar `nDCG@10` sobre datos sintéticos con relevancia como único criterio de optimización, el reranker óptimo es no reordenar —conservar el orden RRF del pool.

Los rerankers sí cambian el conjunto. La hipótesis del audit adversarial era que el reranker estaba devolviendo los mismos ítems que el retriever. F3 la refuta: con un pool rico, `set-change@10` oscila entre 0.527 y 0.821. El comportamiento inerte observado en F2 era un síntoma del pool pobre, no de los rerankers. Este resultado tiene implicaciones para futuros desarrollos: rerankers orientados a diversidad o a objetivos secundarios (revenue, novedad, cobertura de categorías) tienen espacio real para diferenciarse sobre el pool de F3.

NPMI captura complementariedad que el coseno no puede. El test de discriminación commiteado (`tests/thesis/f3-cooccurrence.test.ts`; ejecución observada `npmiHits = 7`, `cosHits = 0`) es una evidencia directa de que las fuentes de co-ocurrencia y las fuentes de similitud semántica capturan señales no solapadas. Esto confirma la premisa arquitectónica de F1 —que la complementariedad de compra (*cross-sell*) no es reducible a la similitud de embedding— y justifica la inclusión de la fuente NPMI como componente irrenunciable del pool multi-fuente.

La ganancia neta de F3 sobre F2 en relevancia pura es modesta a nivel de `nDCG@10` —el baseline RRF del pool alcanza 0.177, frente al 0.152 de F2 en `nDCG@10`—, pero el incremento de recall de pool es el cimiento sobre el que F4 construirá el ranking multi-objetivo: con 84% de las compras futuras presentes en el pool, el optimizador de revenue tiene material suficiente para explorar la frontera de Pareto entre relevancia y margen.

7 Ranking multi-objetivo (F4)

7.1 Motivación: del ranking de relevancia a la negociación de objetivos

El pipeline que F3 construyó es eficaz para maximizar relevancia percibida, medida por `nDCG@10`. Sin embargo, un e-commerce reseller no maximiza relevancia en abstracto: maximiza revenue, custodia la diversidad de su catálogo, cuida la equidad entre vendedores y gestiona la novedad para evitar que el usuario vea siempre los mismos ítems. Estos objetivos no son idénticos y con frecuencia compiten: elevar un ítem de alto precio y margen puede desplazar a un ítem más pertinente para el historial de compra del usuario.

F4 responde a esta tensión con un modelo de ranking multi-objetivo: el score final de cada ítem p para el usuario u es una combinación lineal convexa de funciones de objetivo,

$$s(p \mid u) = \sum_k \lambda_k \cdot f_k(p, u),$$

donde cada f_k representa un objetivo distinto y cada λ_k un peso que el operador puede ajustar. Las funciones de objetivo implementadas son:

- **Relevancia** (f_{rel}): similitud coseno del vector del ítem a los modos del usuario en el espacio $e1_prod2vec$, la misma señal que el reranker de F3.
- **Revenue esperado** (f_{rev}): el producto $P(\text{compra}) \cdot \text{precio} \cdot \text{margen}$, donde la probabilidad de compra se aproxima por la popularidad del ítem en la cohorte demográfica del usuario.
- **Margen bruto** (f_{mar}): margen unitario del ítem, desacoplado del precio para penalizar ítems de bajo margen independientemente de su precio.
- **Diversidad intra-lista** (f_{div}): penalización de similitud media al conjunto ya seleccionado, que promueve cobertura de categorías.
- **Equidad de vendedores** (f_{fair}): señal inversa al índice de Gini de vendedores en la lista emergente, que redistribuye exposición.

La búsqueda de la configuración óptima de λ se realiza mediante un barrido de grilla ($\lambda_{\text{rel}} = 1$ fijo; $\lambda_{\text{rev}} \in \{0, 0.5, 1\}$; $\lambda_{\text{mar}} \in \{0, 0.5\}$; $\lambda_{\text{div}} \in \{0, 0.5\}$; $\lambda_{\text{fair}} \in \{0, 0.5\}$) que genera 24 configuraciones distintas. Sobre las métricas resultantes se construye la frontera de Pareto y se elige un punto de operación.

7.2 Diseño experimental

7.2.1 Universo de evaluación y pool compartido

El experimento opera sobre el mismo universo que F3: 1 998 ítems candidatos, 1 107 casos de evaluación, pool de 200 candidatos por usuario producido por la fusión de cuatro fuentes via RRF. Todos los 24 configuraciones de λ reordenan **el mismo pool**, lo que garantiza que las diferencias de métrica sean atribuibles exclusivamente a la política de ponderación y no a diferencias en el conjunto de candidatos disponibles.

La compra holdout sirve únicamente como verdad de fondo para medir **nDCG@10** y revenue esperado; no es feature de ningún modelo. El experimento es completamente determinista (semilla 42).

7.2.2 Baseline F3-RRF

El punto de referencia es el orden producido por la propia fusión RRF del pool de F3, sin ninguna re-ponderación de objetivos. Sus métricas sobre el conjunto de evaluación son:

Métrica	Valor
nDCG@10 (relevancia)	0.202
revenue@10	29,702.21
diversidad intra-lista@10	0.132
sellerGini@10	0.103

Este baseline representa el máximo de relevancia observado en el estudio: ninguna configuración multi-objetivo lo supera en **nDCG@10**.

7.2.3 Incidencias durante el desarrollo (P0 corregidos)

Tres problemas de primer orden fueron detectados por las revisiones adversariales del arnés y se corrigieron antes de cerrar los resultados:

1. **Conflicto entre margen y revenue.** En el diseño original, el objetivo f_{mar} (margen unitario) peleaba de forma contradictoria contra f_{rev} (revenue esperado = precio $\times$ margen):

elevar el margen unitario a precio bajo reducía el revenue esperado. La solución fue añadir un objetivo f_{rev} explícito que captura el producto conjunto precio-margen-probabilidad de compra, dejando f_{mar} como señal auxiliar opcional.

2. **Runner $O(\text{pool}^2)$.** El scorer original iteraba sobre todos los candidatos para cada ítem al calcular la penalización de diversidad intra-lista, produciendo complejidad cuadrática que bloqueaba el runner con pools grandes. Se introdujo un `limit` de salida anticipada que acota el coste de diversidad a los primeros k ítems ya seleccionados.
3. **Reporte falso de guardrail cumplido.** Un borrador previo describía incorrectamente el punto KPI seleccionado como si satisficiera el guardrail de relevancia $\geq 0.7 \cdot \text{base}$. La revisión final detectó que ninguna de las 24 configuraciones cumple ese umbral (véase la sección de honestidad metodológica) y corrigió el reporte para declararlo infactible.

7.3 Frontera de Pareto y barrido de λ

De las 24 configuraciones evaluadas, **23/24 pertenecen a la frontera de Pareto** cuando se consideran los cuatro objetivos simultáneamente (relevancia, revenue, diversidad, equidad de vendedores). Este resultado confirma que prácticamente toda la grilla de λ genera puntos no dominados: no existe una única configuración que supere a todas las demás en todos los objetivos al mismo tiempo.

La tabla siguiente resume los dos puntos de operación considerados junto con el baseline F3-RRF:

Punto	λ (rel, rev, div)	Relevancia (nDCG@10)	Δ rel	revenue@10	Δ rev
baseline (RRF)	—	0.202	—	29,702	—
knee (min-max) cfg8	rel=1, rev=0.5, div=0	0.081	−59.8%	48,498	+63.3%
revenue-max cfg18	rel=1, rev=1, div=0.5	0.056	−72.4%	52,524	+76.8%

La selección del **punto knee (cfg8)** se realiza mediante normalización min-max sobre las 24 configuraciones: $\text{relN} = (\text{rel} - \text{minRel}) / (\text{maxRel} - \text{minRel})$ e ídem para revenue. El knee maximiza $\min(\text{relN}, \text{revN})$, criterio que es libre de escala y no está dominado por la magnitud bruta del revenue ($\approx 30\,000$ versus relevancia ≈ 0.1). Para **cfg8** se obtiene $\text{relN} = 0.529$ y $\text{revN} = 0.842$.

7.4 Hallazgo central: el trade-off es real y ajustable

El experimento confirma que el trade-off relevancia $\leftrightarrow$ revenue es **real y graduable** (*dial-able*): incrementar λ_{rev} de 0 a 0.5 y de 0.5 a 1 produce incrementos sistemáticos en **revenue@10** a costa de decrementos sistemáticos en **nDCG@10**. El operador puede elegir dónde operar sobre esa curva de intercambio.

El punto knee min-max (**cfg8**) es el compromiso más defendible: conserva el 52.9% del rango de relevancia y el 84.2% del rango de revenue, situándose lejos del extremo degenerado de revenue máximo. En términos absolutos, **cfg8** obtiene +63.3% de **revenue@10** respecto al baseline RRF (48,498 vs 29,702) con una caída de −59.8% en **nDCG@10** (0.081 vs 0.202).

Un hallazgo adicional relevante: **toda configuración rerankeada queda por debajo del baseline RRF en relevancia**. La mejor configuración en relevancia pura (cfg0, rel = 1, todo lo demás = 0) obtiene $nDCG@10 = 0.107$, todavía un -47.1% respecto al baseline 0.202. En este pool sintético, el orden RRF es un óptimo local de relevancia fuerte: cualquier ponderación de objetivos secundarios cuesta relevancia.

7.5 Honestidad metodológica: límites del punto de operación

Esta sección recoge dos advertencias que el lector debe retener al interpretar los resultados anteriores. Son las limitaciones más importantes del capítulo y se exponen sin atenuación.

7.5.1 Guardrail de relevancia infactible: 0/24 configuraciones lo cumplen

El protocolo experimental definía un guardrail de relevancia: $nDCG@10 \geq 0.7 \cdot \text{base} = 0.7 \times 0.202 = 0.141$. Este umbral garantizaría que ningún punto de operación sacrifica más del 30% de la relevancia del baseline.

El guardrail es infactible: ninguna de las 24 configuraciones del barrido lo satisface. La mejor configuración en relevancia pura (cfg0) obtiene $nDCG@10 = 0.107 < 0.141$. La función `pickByKpi` —que selecciona el punto KPI maximizando revenue sujeto al guardrail— no encuentra ningún candidato admisible y **recurrer al máximo global de revenue como solución de fallback**: cfg18 ($nDCG@10 = 0.056$, $\text{revenue}@10 = 52\,524$).

El punto KPI reportado (cfg18) **NO satisface el guardrail de relevancia**. Es el máximo de revenue observado en el barrido, elegido por defecto ante la ausencia de candidatos factibles. Por tanto, no debe interpretarse como un punto de operación de producción deseable: sacrifica el 72.4% de la relevancia del baseline, lo que degradaría gravemente la experiencia de usuario.

El punto de operación recomendado para producción —si se decide desplegar F4— es el knee min-max (cfg8), que representa el compromiso más equilibrado disponible en el espacio explorado, con plena conciencia de que tampoco cumple el guardrail.

7.5.2 Caveat de atribución: el -72.4% conflagra dos efectos distintos

La caída de relevancia del -72.4% entre el baseline RRF y el punto revenue-max (cfg18) no debe leerse íntegramente como el “coste” del objetivo revenue. Ese porcentaje **conflagra dos efectos separados**:

Efecto 1 — Desajuste de baseline (señal única vs fusión de cuatro fuentes). El baseline F3-RRF fusiona cuatro fuentes de candidatos (retrieval coseno, co-ocurrencia NPMI, popularidad por cohorte, exploración aleatoria) mediante RRF. El orden resultante integra señales de relevancia de las cuatro familias. En contraste, el feature de relevancia del scorer multi-objetivo (f_{rel}) es **una sola señal**: la similitud coseno del ítem a los modos del usuario en el espacio `e1\_prod2vec` —aproximadamente equivalente a la fuente retrieval únicamente.

La configuración relevancia-pura (cfg0, $\lambda_{\text{rev}} = 0$, todo lo demás = 0) ya obtiene $nDCG@10 = 0.107$ frente al baseline 0.202: una brecha del -47.1% **sin ponderar revenue en absoluto**. Esta brecha mide el desajuste entre el scorer mono-señal y la fusión RRF, no el trade-off con revenue.

Efecto 2 — Trade-off real relevancia $\leftrightarrow$ revenue. La caída adicional desde cfg0 (0.107) hasta cfg18 (0.056) —unos 0.051 puntos de $nDCG@10$ — sí es atribuible al peso del objetivo revenue. Este es el coste verdadero del trade-off.

En consecuencia, el -72.4% **sobreestima el coste atribuible al objetivo revenue**. Una estimación más honesta del trade-off real tomaría `cfg0` como baseline del scorer en lugar del baseline RRF, o bien incorporaría las señales NPMI y popularidad como features adicionales de f_{rel} , de modo que ambos extremos de la comparación utilizaran el mismo conjunto de información. Esta corrección queda como trabajo futuro.

El hallazgo cualitativo —que el trade-off es real y graduable— **no cambia** con esta corrección: existe una familia de configuraciones que aumenta revenue a costa de relevancia, y la frontera de Pareto es genuina. Lo que cambia es la magnitud del intercambio, que el experimento actual exagera.

7.6 Síntesis

F4 demuestra que el ranking multi-objetivo, con la formulación $s(p \mid u) = \sum_k \lambda_k \cdot f_k$, permite al operador navegar explícitamente el espacio de trade-offs entre relevancia y revenue. La frontera de Pareto (23/24 configuraciones) es real: el dial existe y funciona. El punto de operación recomendado es el knee min-max (`cfg8`, $\lambda_{\text{rel}} = 1$, $\lambda_{\text{rev}} = 0.5$), que obtiene $+63.3\%$ de **revenue@10** respecto al baseline RRF con una caída de -59.8% en **nDCG@10**.

Las dos advertencias metodológicas —guardrail infactible y atribución parcialmente errónea— limitan la transferibilidad directa a producción pero no invalidan el hallazgo central: el mecanismo de ponderación λ es el instrumento correcto para negociar objetivos de negocio en un sistema de ranking personalizado, y su calibración requiere un piloto A/B con métricas de conversión reales antes de cualquier despliegue.

8 Discusión

Este capítulo integra los hallazgos de los cuatro factores (F1–F4) del programa de tesis en una lectura transversal. El objetivo es doble: identificar los patrones que atraviesan múltiples fases y extraer las lecciones de diseño que trascienden cada experimento individual. Se dedica una sección prominente a los hallazgos negativos, que tienen valor epistémico igual o superior a los positivos. Se cierran con las limitaciones del enfoque y las amenazas a la validez interna y externa.

8.1 Qué funcionó: convergencia entre fases

8.1.1 Los embeddings comportamentales son la capa correcta para relevancia

F1 estableció que los embeddings entrenados sobre co-compras superan al texto puro en recuperación de relevancia, con una ratio de $\approx 2.7\times$ en **nDCG@10** para `e1_prod2vec` y $\approx 3.3\times$ para la fusión híbrida `e2_hybrid`. La fusión híbrida `e2_hybrid` —que combina el espacio Voyage de texto (1024 dimensiones) y el espacio Prod2Vec comportamental (64 dimensiones) mediante fusión de score— demostró que la señal comportamental no sustituye a la textual sino que la complementa: el híbrido supera a ambos componentes por separado. Esta complementariedad persiste a lo largo de todo el programa: el espacio `e1_prod2vec` se convierte en la fuente de relevancia de referencia en F2, F3 y F4, y sus límites —concretamente, su incapacidad para capturar complementariedad— motivan directamente la segunda fuente NPMI del pool de F3.

El resultado de F1 responde a una pregunta que el pipeline original no podía responder: ¿qué tipo de señal captura mejor la preferencia del usuario en un catálogo de reventa sin historial de búsqueda explícita? La respuesta es inequívoca: la geometría del espacio de co-compra es más cercana a la preferencia declarada que la semántica del título del producto.

8.1.2 La representación multi-vector del usuario, especialmente en el segmento de regalo

F2 mostró que modelar al usuario como distribución de intereses (múltiples medoides en el espacio `e1_prod2vec`) supera al vector único en todos los segmentos evaluados. La ganancia global es de aproximadamente el 51% relativo en `nDCG@10`. La lección más nítida es la asimetría del efecto según el tipo de sesión: en autoconsumo con intereses unimodales el beneficio existe pero es moderado; en sesiones de regalo la diferencia es cualitativa. Con el vector único de comprador, el `nDCG@10` en el peor segmento de regalo cae a valores cercanos a cero; el vector efímero de destinatario lo eleva en un factor de aproximadamente $12\times$.

Esta asimetría tiene una interpretación arquitectónica directa: la representación multi-vector con modelo de regalo no es una optimización marginal sobre el vector único, sino una bifurcación de paradigma. El vector único asume que todos los ítems adquiridos revelan la preferencia futura del comprador; el modelo de F2 separa las compras de autoconsumo —que deben alimentar el perfil permanente— de las compras para un tercero, que solo deberían activarse durante la sesión en curso. La separación entre perfil del comprador y vector efímero de destinatario es una decisión de diseño justificada empíricamente, no una complejidad añadida sin respaldo.

8.1.3 El pool multi-fuente aproximadamente duplica el recall de candidatos

F3 demostró que la ganancia principal del factor reside en el pool de candidatos, no en el reranker. El paso de un pool de 30 candidatos (top-30 coseno de F2) a un pool multi-fuente de 200 candidatos —construido con cuatro fuentes fusionadas por RRF— eleva el recall del pool de 0.410 a 0.839: $\sim 2\times$ más compras futuras cubiertas. La consecuencia práctica es directa: el 84% de las compras que el usuario realizará ya están disponibles en el pool para que el reranker las eleve, frente al 41% de F2. Este porcentaje es el techo real de cualquier reranker sobre ese pool; elevar el techo tiene más impacto que mejorar el reranker dentro del techo existente.

La fuente NPMI mereció atención especial: en un test de discriminación sobre casos de regalo con verdad de fondo conocida, recuperó siete complementos que la similitud coseno no encontró en absoluto. Esta evidencia puntual confirma la premisa arquitectónica establecida en F1: la complementariedad de compra es una señal ortogonal a la similitud semántica, y las dos fuentes deben coexistir en el pool.

8.1.4 La frontera de Pareto hace explícito el trade-off de negocio

F4 demostró que el ranking multi-objetivo, formulado como combinación lineal convexa $s(p | u) = \sum_k \lambda_k \cdot f_k(p, u)$, permite al operador navegar explícitamente la tensión entre relevancia y revenue. La frontera de Pareto sobre las 24 configuraciones evaluadas es genuina: 23/24 son no dominadas cuando se consideran los cuatro objetivos simultáneamente, lo que indica que no existe una configuración que gane en todo. El dial λ funciona: incrementar λ_{rev} de 0 a 0.5 y de 0.5 a 1 produce incrementos sistemáticos en `revenue@10` a costa de decrementos sistemáticos en `nDCG@10`.

El valor de este hallazgo no está en los números absolutos —que dependen de los precios y márgenes sintéticos del simulador— sino en la lección arquitectónica: la frontera de Pareto convierte una decisión implícita (el pipeline de relevancia pura ya toma una posición tácita de $\lambda_{\text{rev}} = 0$) en una decisión explícita y ajustable por el operador sin reentrenamiento. Esta es la contribución de F4 a la práctica del sistema.

8.2 Los hallazgos negativos como contribución científica

Los resultados negativos de este programa son, en conjunto, tan informativos como los positivos. Cada uno de ellos delimita el alcance de una técnica y señala dónde debe invertirse el esfuerzo en lugar de dónde ya se invirtió. Esta sección los recoge sin atenuación, argumentando por qué merecen protagonismo propio.

8.2.1 Los embeddings no recuperan complementos (F1)

La columna `complement-recall@10` del experimento F1 es prácticamente cero para los seis espacios vectoriales evaluados, incluyendo los comportamentales. Un ítem complementario —la alfombrilla del ratón, la funda del teléfono, la pila del mando— no aparece en el top-10 de ningún retriever de similitud vectorial, independientemente de si ese retriever fue entrenado sobre texto o sobre co-compras.

Este resultado negativo delimitó la arquitectura de todo lo que siguió. Si el coseno no captura complementariedad, la solución no es mejorar el embedding sino introducir una fuente de señal diferente: la co-ocurrencia de compra cuantificada por NPMI. El hallazgo negativo de F1 es la justificación directa de la segunda fuente del pool de F3, que recuperó siete complementos donde la similitud coseno recuperó cero. Sin ese resultado negativo honestamente reportado, la arquitectura multi-fuente carecería de motivación empírica y habría que aceptarla por argumento de autoridad.

La lección general es que los embeddings son la herramienta correcta para relevancia y descubrimiento, pero no para cross-sell. El cross-sell vive en el grafo de co-ocurrencia o en puntuaciones NPMI sobre pares de productos, no en el espacio vectorial de similitud. La claridad de esta separación arquitectónica es consecuencia directa del hallazgo negativo.

8.2.2 Ningún reranker supera a la fusión RRF en relevancia pura (F3)

El hallazgo negativo central de F3 es que el orden de relevancia producido por la propia fusión RRF —el mecanismo que construye el pool— es el mejor ranker disponible en el experimento, con `nDCG@10` = 0.177. Todos los rerankers evaluados (LTR, MMR, cross-encoder MaxSim y LLM listwise con DeepSeek) quedan por debajo de ese umbral. El LLM listwise alcanza `nDCG@10` = 0.170, virtualmente idéntico al baseline RRF, pero sin mejora neta.

Este hallazgo es contraintuitivo: la intuición general en la literatura de recuperación es que un reranker más sofisticado produce resultados superiores. La explicación en este caso particular es que el pool de candidatos ya integra señales de cuatro fuentes complementarias, y el orden RRF resultante agrega esa información de forma que los rerankers evaluados no pueden mejorar. El `retrievalScore` domina los pesos del modelo LTR (peso 6.82 frente a 0.29 de `npmiScore`), lo que indica que el modelo aprendido delega en la puntuación RRF del pool, replicando aproximadamente al baseline.

La implicación para el diseño futuro es que el esfuerzo no debe dirigirse a rerankers de relevancia más sofisticados —el techo del pool ya es alto— sino a rerankers orientados a objetivos secundarios: diversidad, revenue, novedad, equidad de vendedores. En ese dominio, los rerankers sí tienen espacio para diferenciarse, como demuestra el `set-change@10` de 0.821 del cross-encoder: los ítems que propone son cualitativamente distintos a los del baseline RRF, aunque su relevancia posicional sea inferior.

8.2.3 El guardrail de relevancia de F4 es infactible: 0/24 configuraciones

El protocolo experimental de F4 definía un guardrail de relevancia: $\text{nDCG@10} \geq 0.7 \times \text{base} = 0.141$, pensado para garantizar que ningún punto de operación sacrificara más del 30% de la relevancia del baseline. El guardrail resultó infactible: ninguna de las 24 configuraciones del barrido de λ lo satisface. La mejor configuración en relevancia pura (`cfg0`, $\lambda_{\text{rev}} = 0$) obtiene $\text{nDCG@10} = 0.107$, por debajo del umbral de 0.141.

Este resultado tiene dos capas que deben distinguirse para leerlo correctamente.

La primera capa es el hallazgo genuino sobre el trade-off: el mecanismo de ponderación λ es eficaz para aumentar revenue pero no puede hacerlo sin coste de relevancia, y el coste es alto dentro del espacio de búsqueda explorado. Este es un hallazgo real y útil: cualquier despliegue en producción debe asumir que mover el dial hacia revenue tiene consecuencias apreciables en la calidad de las listas.

La segunda capa es el caveat de atribución. La caída de relevancia observada entre el baseline RRF (0.202) y la mejor configuración en relevancia pura (`cfg0`, 0.107) —una brecha del -47.1% — no es atribuible al objetivo revenue sino a un desajuste de baseline: el baseline F3-RRF fusiona cuatro fuentes de señal, mientras que el feature de relevancia del scorer multi-objetivo (f_{rel}) usa únicamente similitud coseno en el espacio `e1_prod2vec`, aproximadamente equivalente a una sola fuente. El scorer mono-señal ya cede relevancia al baseline multi-señal antes de incorporar revenue.

Este conflató entre trade-off real y desajuste de baseline no invalida el hallazgo central —el dial de revenue funciona y la frontera de Pareto es genuina— pero sí sobreestima el coste del trade-off. Una medición más honesta requeriría que f_{rel} integrara las mismas señales que el baseline RRF (NPMI, popularidad por cohorte), de modo que la comparación sea entre configuraciones equivalentes en información disponible. Esa corrección queda como trabajo futuro.

El hallazgo negativo aquí no es un fracaso técnico sino una advertencia metodológica: el diseño del baseline de comparación determina la magnitud atribuida al trade-off, y la comparación solo es justa cuando ambos extremos usan la misma información de base.

8.2.4 Por qué los negativos son tan valiosos como los positivos

Estos tres hallazgos negativos —`complement-recall@10` ≈ 0 en todos los espacios, ningún reranker supera a RRF en relevancia, guardrail infactible con caveat de atribución— tienen en común que redirigen el esfuerzo de ingeniería hacia donde sí existe potencial de mejora. La complementariedad de compra exige co-ocurrencia o NPMI, no embeddings más potentes. El techo de relevancia del pool exige ampliar el pool o mejorar las fuentes, no añadir rerankers más complejos. La subestimación del feature de relevancia en F4 exige incorporar señales multi-fuente al scorer antes de interpretar la magnitud del trade-off como un hecho definitivo.

Sin estos hallazgos negativos, los recursos de desarrollo tenderían erróneamente hacia embeddings con mayor dimensionalidad, rerankers LLM más caros o guardrails con umbrales más permisivos. Con ellos, el diseño puede tomar decisiones informadas: invertir en NPMI para cross-sell, ampliar el pool para relevancia, y enriquecer el feature de relevancia para una evaluación justa del trade-off en F4.

8.3 Limitaciones del enfoque

8.3.1 Dataset sintético: validez externa pendiente

Todos los experimentos operan sobre un simulador de marketplace con verdad de fondo conocida. La ventaja es metodológica: las preferencias latentes del usuario, las relaciones de complementariedad y la compra holdout son generadas por el sistema y conocidas de antemano, lo que permite medir el lift de cada componente sin confusión con sesgos de exposición o popularidad. La desventaja es la misma: los resultados son válidos para el simulador, y su transferibilidad a tráfico de producción real es una hipótesis no validada.

En particular, el simulador podría asignar señales más limpias a los complementos NPMI y a las sesiones de regalo que las que existirían en un dataset de sesiones reales, donde la intención del usuario es ruidosa y los patrones de co-compra son más dispersos. El cross-check sobre un dataset público de sesiones reales (`thesis:public`) está diseñado pero no ejecutado: es el siguiente paso natural para evaluar la robustez de los hallazgos.

8.3.2 El detector de regalo es heurístico con precisión imperfecta

La detección de sesiones de regalo en F2 es heurística: utiliza coherencia demográfica entre los ítems de la sesión corriente y la cohorte del comprador, sin señal explícita del usuario. Su precisión es ~ 0.43 , con recall de 0.387 y F1 de 0.407. Esto significa que aproximadamente el 57% de las activaciones del vector efímero corresponden a sesiones que no son de regalo (falsos positivos), y el detector no detecta el 61% de las sesiones que sí lo son (falsos negativos).

Las consecuencias son asimétricas: los falsos positivos introducen ruido en sesiones de autoconsumo —activando el vector efímero cuando no debería— y los falsos negativos degradan la calidad de las sesiones de regalo al no activarlo cuando sí debería. La ganancia medida en F2 sobre `nDCG@10` y `recipient-fit@10` es el efecto neto del modelo completo incluyendo sus errores de clasificación. El beneficio de la representación multi-vector es posiblemente mayor de lo que los números indican; el modelo de regalo podría extraer más valor con un detector más preciso, por ejemplo entrenado sobre señales explícitas del usuario o con inferencia demográfica supervisada.

8.3.3 El baseline de relevancia de F4 usa una única señal

El feature de relevancia del scorer multi-objetivo en F4 (f_{rel}) es la similitud coseno al espacio `e1_prod2vec`, equivalente a una sola fuente de las cuatro que componen el baseline RRF. Esta asimetría introduce un desajuste estructural: el baseline tiene acceso a más información de relevancia que el scorer, lo que hace que la caída de `nDCG@10` observada al introducir objetivos de revenue esté parcialmente inflada por ese desajuste y no sea íntegramente atribuible al trade-off. La magnitud real del trade-off relevancia $\leftrightarrow$ revenue requiere una medición con baseline y scorer equivalentes en información, lo que no está implementado en este estudio.

8.3.4 Costo y latencia del reranker LLM listwise

El reranker LLM listwise (DeepSeek) operó sobre un subconjunto de 120 casos por razones de costo y latencia. La evaluación limitada introduce incertidumbre estadística sobre sus resultados, aunque los hallazgos cualitativos son consistentes con los del resto de rerankers. La escalabilidad del reranker LLM a la totalidad del catálogo (1998 ítems, pool de 200) en tiempo de respuesta compatible con producción no ha sido medida. Su costo por consulta en producción real dependería

de la disponibilidad y precio del proveedor LLM, una variable económica externa al diseño del sistema.

8.3.5 Escala del estudio

El universo del estudio es de aproximadamente 2000 productos y 1100 casos de evaluación. Esta escala es suficiente para detectar diferencias de calidad estadísticamente robustas entre métodos, pero es pequeña respecto a catálogos de producción reales que pueden contener cientos de miles de ítems. Los efectos de escala —costos de indexación, latencia de retrieval, dispersión de co-ocurrencias NPMI en catálogos grandes— no han sido evaluados y constituyen una incertidumbre de transferibilidad.

8.4 Amenazas a la validez y mitigaciones

8.4.1 Amenaza 1: el ground-truth sintético podría favorecer ciertos métodos

El simulador genera las preferencias latentes del usuario y las relaciones de complementariedad. Si las reglas de generación son consistentes con los supuestos de los métodos evaluados —por ejemplo, si la co-ocurrencia NPMI del simulador está diseñada para ser recuperable por el grafo de co-compra— el experimento podría sobreestimar la ventaja de los métodos que usan NPMI respecto a métodos alternativos que no dispusieran de esa señal.

Las mitigaciones adoptadas son: las reglas del simulador fueron definidas antes de medir cualquier resultado (protocolo de pre-registro implícito); las comparaciones se realizan mediante ablations controladas donde todos los métodos comparten el mismo universo de ítems y los mismos casos de evaluación; y el cross-check sobre un dataset público de sesiones reales (`thesis:public`) está previsto como test de robustez externo. Hasta que ese cross-check no se ejecute, la amenaza permanece abierta.

8.4.2 Amenaza 2: mezcla de escalas entre fases

F0 operó sobre aproximadamente 400 usuarios; F1–F4 operan sobre el universo de 1999 ítems con 1098–1107 casos de evaluación. Mezclar estas escalas en una tabla comparativa produciría comparaciones sin sentido. La mitigación fue no hacerlo: cada fase se compara internamente, y cuando se hace referencia a fases anteriores se indica explícitamente la escala y el espacio de ítems correspondiente.

8.4.3 Amenaza 3: variabilidad por semilla

Los experimentos deterministas (semilla 42 en F4, siembra del simulador en todas las fases) garantizan reproducibilidad exacta de los resultados, pero una única semilla no permite cuantificar la varianza de las métricas bajo distintas realizaciones del generador. La variabilidad real de los resultados ante distintas configuraciones del simulador es una incertidumbre no resuelta. Su mitigación parcial es que las conclusiones cualitativas son robustas a pequeñas variaciones en las métricas; las conclusiones cuantitativas exactas deben leerse con la reserva de que corresponden a una única realización.

9 Conclusión y trabajo futuro

9.1 Conclusión

El programa de tesis F1–F4 partió de una pregunta motivacional honesta: ¿cuándo y cuánto ayuda la personalización profunda en un e-commerce reseller, y dónde no ayuda? La respuesta que surge del arco experimental es matizada y, precisamente por ello, más útil para el diseño del sistema.

9.1.1 Lo que el arco experimental demostró

La representación importa, y su elección no es obvia. F1 estableció que los embeddings comportamentales superan al texto puro en recuperación de relevancia, con una ratio de aproximadamente $2.7\times$ para `e1_prod2vec` y $3.3\times$ para la fusión `e2_hybrid` en `nDCG@10`. Pero el mismo experimento demostró que esos embeddings no recuperan complementos: la columna `complement-recall@10` es prácticamente cero en todos los espacios evaluados. La conclusión arquitectónica —que la relevancia y el cross-sell requieren señales fundamentalmente distintas— no habría sido alcanzable sin medir ambas con la misma rigurosidad.

El usuario como distribución, no como punto, se justifica empíricamente. F2 demostró que el modelo multi-vector con detección de regalo supera al vector único en todos los segmentos evaluados. La ganancia más pronunciada aparece exactamente donde la teoría predice que debería aparecer: en las sesiones de regalo, donde el vector único de comprador colapsa a valores de `nDCG@10` cercanos a cero, mientras que el vector efímero de destinatario eleva la métrica en un factor de aproximadamente $12\times$ en el peor segmento. Esta consistencia entre predicción teórica y resultado empírico da confianza en el diseño del modelo, a pesar de las imperfecciones del detector heurístico.

El cross-sell vive en la co-ocurrencia, no en el coseno. F3 confirmó que la fuente NPMI recupera complementos que la similitud vectorial no puede recuperar. El pool multi-fuente aproximadamente duplica el recall de candidatos respecto al retriever coseno de F2, y esa ampliación de recall es el avance principal de F3. La lección de diseño es que el esfuerzo de ingeniería en la etapa de retrieval debe distribuirse entre fuentes de señal ortogonales, no concentrarse en embeddings más potentes dentro de una única señal.

El ranking negocia múltiples objetivos, y la frontera de Pareto es el instrumento correcto para esa negociación. F4 demostró que la combinación lineal convexa con pesos λ ajustables convierte una decisión implícita —el pipeline de relevancia pura ya elige tácitamente $\lambda_{\text{rev}} = 0$ — en una decisión explícita y controlable por el operador. La frontera de Pareto (23/24 configuraciones no dominadas) es genuina: el dial existe y funciona, y su calibración final requiere un piloto A/B con métricas de conversión reales.

Los hallazgos negativos son contribuciones de primera clase. El `complement-recall@10` ≈ 0 de F1, la inferioridad de todos los rerankers frente al baseline RRF en F3, y la infactibilidad del guardrail en F4 —con el correspondiente caveat de atribución sobre la señal única de relevancia— no son fracasos a minimizar en las conclusiones. Son los resultados que delimitan el espacio de soluciones eficaces, señalan dónde poner el esfuerzo en lugar de dónde ya se puso, y constituyen evidencia reproducible sobre las fronteras de lo que los embeddings y los rerankers de relevancia pueden y no pueden hacer en este dominio.

9.1.2 La tesis en una frase

La personalización profunda se justifica cuando (a) la representación del usuario captura la multimodalidad real de sus intereses y la naturaleza efímera de las sesiones de regalo, (b) el cross-sell se modela donde vive —en la co-ocurrencia de compra, no en la similitud coseno de embeddings— y (c) el ranking negocia explícitamente múltiples objetivos del negocio mediante una frontera de Pareto ajustable sin reentrenamiento; todo ello medido con rigor sobre un arnés con holdout temporal y reportando los límites con la misma honestidad que las victorias.

9.2 Trabajo futuro

Los hallazgos del programa de tesis sugieren seis líneas de trabajo futuro, ordenadas por prioridad metodológica.

9.2.1 Cross-check sobre dataset público de sesiones (validez externa)

La amenaza más importante sobre los resultados actuales es que todo el programa opera sobre datos sintéticos. El cross-check sobre un dataset público de sesiones reales de e-commerce, para el que el adaptador `thesis:public` ya está implementado, es el siguiente paso inmediato. El objetivo es verificar que las conclusiones cualitativas —ventaja de embeddings comportamentales sobre texto, beneficio de multi-vector en sesiones de regalo, contribución ortogonal de co-ocurrencia NPMI— se replican en datos con ruido real, distribuciones de popularidad no sintéticas y patrones de compra conjunta emergentes en lugar de simulados. Los resultados cuantitativos exactos cambiarán; lo que se quiere verificar es si el orden cualitativo entre métodos se preserva.

9.2.2 Item2Vec y two-tower entrenados con clics y compras reales

El embedding `e3_two_tower` quedó considerablemente por debajo de `e1_prod2vec` en F1, posiblemente por insuficiencia de datos de entrenamiento en el simulador. En producción, el pipeline dispone o dispondrá de señal real de clics y compras, que es precisamente la señal que item2vec y los modelos two-tower necesitan para generalizar en catálogos de cola larga. El fine-tuning de un two-tower con eventos de compra reales y muestreo negativo in-batch —con corrección logQ para descontar el sesgo de popularidad— es la vía más directa para superar las limitaciones del Prod2Vec entrenado sobre historial limitado. La evaluación debe reproducir el mismo protocolo de holdout temporal para mantener la comparabilidad con los resultados de F1.

9.2.3 ColBERT real vía API multilingüe

El reranker cross-encoder MaxSim de F3 implementó una aproximación a late interaction con embeddings estáticos, que obtuvo el peor resultado en `nDCG@10` aunque el mayor en `set-change@10`. Un ColBERT real con codificación token-a-token en un modelo multilingüe —relevante para un catálogo con títulos en inglés, español y chino— podría capturar relevancia contextual que el MaxSim estático no puede. La API multilingüe de Voyage ya está disponible en el entorno; la integración de ColBERT real requiere adaptar el esquema de pool para almacenar representaciones multi-token por ítem, lo que tiene implicaciones de latencia y costo que deben medirse antes de cualquier decisión de despliegue.

9.2.4 Bandit contextual para ajustar λ por usuario

F4 establece la frontera de Pareto y propone un punto de operación fijo (el knee min-max `cfg8`) para todos los usuarios. Esta elección es una aproximación razonable pero subóptima: usuarios con alta propensión a compra de artículos de alto valor podrían tolerar —o incluso preferir— una configuración más orientada a revenue; usuarios exploradores podrían beneficiarse de más diversidad. Un bandit contextual que ajuste λ por usuario en función de señales de contexto (historial de compra, cohorte demográfica, momento de sesión) convertiría el peso único del sistema en un peso personalizado sin necesidad de reentrenamiento global. La implementación puede reutilizar el mismo scorer multi-objetivo de F4; lo que varía es el mecanismo que elige λ para cada solicitud.

9.2.5 Features de relevancia multi-señal para un baseline justo en F4

El caveat de atribución identificado en F4 —que el scorer mono-señal ($f_{\text{rel}} = \text{similitud coseno a } \mathbf{e1_prod2vec}$) ya cede el -47.1% de relevancia frente al baseline RRF antes de incorporar revenue— no invalida el hallazgo central, pero sí impide medir con precisión cuánta relevancia cuesta realmente el trade-off. La corrección directa es enriquecer f_{rel} con las mismas señales que componen el baseline RRF: puntuación NPMI de co-ocurrencia, popularidad por cohorte, y eventualmente la señal aleatoria de exploración. Con un scorer que integre las cuatro fuentes, la diferencia entre `cfg0` (relevancia pura) y el baseline RRF desaparecería, y la caída atribuible a λ_{rev} mediaría únicamente el trade-off real. Esta corrección cierra el único caveat de atribución abierto del programa.

9.2.6 Ejecución del piloto A/B en producción

El capítulo siguiente (`10-plan-piloto.md`) diseña sin ejecutar un experimento controlado en producción. La ejecución de ese piloto es el horizonte natural del programa de tesis: convertir los hallazgos offline en evidencia de impacto en métricas de negocio reales (tasa de conversión, revenue por sesión, retención de usuarios). El piloto está diseñado para validar separadamente la ganancia de F2 (multi-vector con regalo) y la configuración knee de F4 ($\lambda_{\text{rel}} = 1, \lambda_{\text{rev}} = 0.5$), con poder estadístico suficiente para detectar efectos de la magnitud observada en el simulador. Su ejecución requiere instrumentación de clics y compras en producción, que es la señal que retroalimentará también el fine-tuning del two-tower mencionado anteriormente.

10 Plan de piloto A/B (diseño, no ejecutado)

Nota de alcance. Este capítulo diseña un experimento controlado para validar en producción las contribuciones del programa de tesis F1–F4. El piloto **no ha sido ejecutado**: los datos de tráfico real, las tasas de conversión y los tamaños de efecto sobre métricas de negocio son, a la fecha de este escrito, desconocidos. Lo que sigue es un protocolo experimental —hipótesis, diseño de asignación, KPIs, guardrails y fases de rollout— cuya ejecución constituye el horizonte natural del trabajo futuro descrito en el capítulo anterior.

10.1 Justificación: por qué el piloto es la validación que falta

El programa de tesis F1–F4 operó íntegramente sobre un simulador de marketplace con verdad de fondo conocida y un arnés de evaluación con holdout temporal. Esta configuración permite

controlar variables y aislar efectos con precisión difícil de alcanzar en sistemas de producción, pero introduce una amenaza de validez que ningún resultado offline puede disipar: los datos son sintéticos. Las distribuciones de popularidad, los patrones de compra conjunta y los perfiles de usuario fueron generados por un modelo probabilístico, no observados en el comportamiento de compradores reales.

El piloto A/B en producción es, por tanto, la única forma de cerrar este ciclo. Tres hallazgos del programa offline motivan específicamente el diseño del experimento:

1. **El pool multi-fuente aproximadamente duplica el recall de candidatos** (de 0.410 a 0.839) respecto al retriever coseno de F2. Si este efecto se replica en producción, las listas de feed serán cualitativamente distintas, con mayor presencia de complementos y de ítems de cola larga.
 2. **El trade-off de F4 es real y graduable.** El punto knee min-max (`cfg8`, $\lambda_{\text{rel}} = 1$, $\lambda_{\text{rev}} = 0.5$) obtiene +63.3% de `revenue@10` respecto al baseline RRF con una caída de -59.8% en `nDCG@10` sobre datos sintéticos. El impacto de esa caída de relevancia sobre el comportamiento real de compra —¿cuánto engagement pierde el usuario? ¿se compensa con el incremento de revenue?— es una pregunta empírica que sólo el piloto puede responder.
 3. **El detector de regalo es imperfecto** (precisión ≈ 0.43 , $F1 \approx 0.41$). En producción, la señal demográfica puede complementarse con señales de comportamiento en sesión (consultas de búsqueda, hora del día, duración de la sesión) que el simulador no genera. El piloto permite observar si la ganancia del vector efímero de destinatario — $\approx 5\text{--}12\times$ en `nDCG@10` sobre segmentos de regalo en el estudio offline— se transfiere al entorno real pese a los errores del detector.
-

10.2 Qué se promueve a producción bajo feature flags

Las cuatro contribuciones del programa de tesis se agrupan en un único tratamiento para el piloto. Todas ellas viven en `src/sectors/d-personalization/` y se activarán mediante feature flags independientes, lo que permite tanto el rollout gradual como la desactivación selectiva ante un guardrail cruzado.

10.2.1 FF-1: espacio de retrieval `e2_hybrid`

El retrieval coseno en producción usa actualmente `e1_prod2vec` como único espacio. `e2_hybrid` fusiona las puntuaciones de texto (Voyage AI, 1024 dimensiones) y comportamental (`e1_prod2vec`, 64 dimensiones), sin reentrenamiento conjunto, operando sobre puntuaciones normalizadas para evitar que la mayor dimensionalidad del espacio textual domine la similitud resultante. F1 sitúa `e2_hybrid` como el espacio con mayor `nDCG@10` y `Recall@10` entre los seis evaluados. Su adopción amplía la cobertura semántica del retriever al coste de una llamada adicional al índice de texto.

10.2.2 FF-2: representación multi-vector del usuario con vector efímero de destinatario

La representación de usuario se eleva de un vector único a una distribución de medoides (clustering aglomerativo coseno sobre el historial, con el número de modos emergiendo de los datos). Para sesiones detectadas como regalo, el sistema construye un vector efímero de destinatario —calculado únicamente a partir de los ítems de la sesión corriente en curso y desechado al concluirla— sin alterar

el perfil permanente del comprador. Ante baja confianza en la detección de regalo (score heurístico inferior a un umbral configurable), el sistema degrada graciosamente a modo de autoconsumo (**self-mode**), evitando que los errores de clasificación contaminen el feed.

10.2.3 FF-3: pool de candidatos multi-fuente vía RRF

El pool de candidatos se amplía de los 30 ítems top-coseno actuales a un pool de 200 ítems producido por la fusión RRF de cuatro fuentes:

- Retrieval por similitud coseno en **e2_hybrid** (80 candidatos).
- Co-ocurrencia NPMI del último ítem observado (complementos de compra conjunta).
- Popularidad por cohorte demográfica (red de seguridad para historiales cortos).
- Exploración aleatoria sembrada (diversidad y cobertura de cola larga).

La fusión RRF garantiza que ninguna fuente domine completamente la composición del pool. El coste adicional de recuperación por las fuentes NPMI y de popularidad es local (tablas en base de datos, sin llamadas externas al agregador), por lo que no incrementa el costo por solicitud de los agregadores de catálogo.

10.2.4 FF-4: reranking en el punto knee de la frontera multi-objetivo

El reranker multi-objetivo opera con la configuración **cfg8** del barrido de F4: $\lambda_{\text{rel}} = 1$, $\lambda_{\text{rev}} = 0.5$, $\lambda_{\text{div}} = 0$. Este punto knee min-max fue seleccionado como el compromiso más equilibrado entre relevancia y revenue esperado disponible en el espacio explorado por el barrido. Se elige deliberadamente el knee y **no** el extremo de revenue máximo (**cfg18**, $\lambda_{\text{rev}} = 1$): el trade-off de F4 muestra que maximizar revenue en el estudio offline sacrifica el 72.4% de la relevancia del baseline, una magnitud que, trasladada a producción, previsiblemente dañaría el engagement y la retención del usuario a medio plazo.

La estrategia de producción es conservadora por diseño: **empezar en el knee y moverse sobre la frontera de Pareto únicamente si el engagement de largo plazo aguanta**. Si el piloto muestra que el engagement (profundidad de sesión, CTR) se mantiene estable o mejora, el ajuste posterior del peso λ_{rev} puede delegarse a un bandit contextual (línea de trabajo futuro descrita en el capítulo anterior), sin necesidad de reentrenamiento global del sistema.

10.3 Diseño experimental

10.3.1 Condiciones del experimento

El piloto compara dos condiciones:

- **Condición A (control):** el pipeline actual en producción. Retrieval coseno sobre **e1_prod2vec**, representación de usuario con vector único, pool de 30 candidatos, ranking por relevancia/RRF sin ponderación de objetivos de negocio.
- **Condición B (tratamiento):** el pipeline completo de las contribuciones F1–F4, activado mediante los cuatro feature flags descritos en la sección anterior. Espacio **e2_hybrid**, representación multi-vector con vector efímero de destinatario (con degradación graceful ante baja confianza), pool multi-fuente de 200 ítems vía RRF, y reranking en el punto knee ($\lambda_{\text{rel}} = 1$, $\lambda_{\text{rev}} = 0.5$).

10.3.2 Unidad de asignación y estratificación

La unidad de asignación es el **usuario** (no la sesión ni la solicitud). La asignación a control o tratamiento se realiza mediante **hash determinista** del identificador de usuario, lo que garantiza que:

- (a) un mismo usuario ve siempre la misma condición durante toda la duración del piloto, evitando la contaminación por exposición cruzada;
- (b) la asignación es reproducible y auditable sin almacenar estado de experimento por solicitud;
- (c) la fracción de usuarios en cada condición es estable y no depende del orden de llegada.

Los usuarios se estratifican por el segmento de modo de sesión —autoconsumo (**self**) vs regalo (**gift**, según el detector con degradación graceful)— antes de la asignación, de forma que ambas condiciones reciben proporciones equivalentes de cada segmento. Esto es especialmente importante dado que F2 muestra efectos cualitativamente distintos en los dos segmentos: ignorar la estratificación podría enmascarar una ganancia real en sesiones de regalo o una regresión en sesiones de autoconsumo.

10.3.3 Métrica primaria y KPIs secundarios

La **métrica primaria** del piloto es el **revenue o GMV por sesión**: el valor monetario bruto de las compras realizadas durante la sesión, normalizado por el número de sesiones activas de cada condición. Esta métrica captura directamente el objetivo de negocio del sistema: en un e-commerce reseller sin stock físico, cada compra completada es el evento de valor real para el operador.

Los KPIs secundarios son:

- **CTR del feed** (tasa de clic sobre los ítems mostrados en la lista personalizada): mide si el tratamiento produce un feed que atrae la atención del usuario. Un CTR degradado es señal temprana de pérdida de relevancia percibida.
- **CVR (tasa de conversión)**: fracción de sesiones con al menos una compra completada. Complementa el revenue por sesión: un aumento de revenue por sesión puede originarse en un aumento de CVR, en un aumento del ticket medio, o en ambos; distinguirlos orienta la interpretación.
- **Profundidad de sesión** (número de páginas de feed vistas por sesión): proxy de engagement. Una caída de profundidad en el tratamiento indicaría que el usuario encuentra lo que busca más rápido (señal positiva) o que abandona el feed antes de convertir (señal negativa); debe leerse junto con el CTR y el CVR.
- **Tasa de uso del fallback del reranker LLM**: fracción de solicitudes en que el reranker LLM no puede completar el reranking y recurre al orden de entrada como fallback. Un incremento de esta tasa en el tratamiento señala degradación de disponibilidad del reranker o prompts que superan el límite de contexto, y activa la revisión del módulo antes de escalar.

El análisis de KPIs se desagregará por segmento **self** vs **gift** para detectar si el tratamiento beneficia a un segmento a expensas del otro.

10.3.4 Cálculo del tamaño muestral

No disponemos, a la fecha de este diseño, de estimaciones de tráfico real para el sistema en producción. Por tanto, no se especifican números concretos de usuarios o semanas. En su lugar, se describe el **procedimiento de cálculo** que deberá ejecutarse con los datos de tráfico reales antes de iniciar el piloto:

1. **Definir el MDE (Minimum Detectable Effect).** El equipo debe acordar el efecto mínimo en revenue por sesión que justifica adoptar el tratamiento —por ejemplo, un incremento relativo de $X\%$ sobre la media de control. El MDE debe ser clínicamente relevante para el negocio, no arbitrariamente pequeño.
2. **Estimar la varianza de la métrica primaria.** Con datos históricos de revenue por sesión en producción, calcular la desviación estándar σ de la distribución. El revenue por sesión en e-commerce tiene típicamente una distribución de cola pesada (muchas sesiones sin compra, pocas con compras de alto valor); puede ser necesario aplicar una transformación logarítmica o usar un test no paramétrico.
3. **Aplicar la fórmula estándar de potencia.** Para un test bilateral con nivel de significación $\alpha = 0.05$ y potencia $1 - \beta = 0.80$, el tamaño muestral por condición es aproximadamente:

$$n \approx \frac{2(z_{1-\alpha/2} + z_{1-\beta})^2 \cdot \sigma^2}{\delta^2}$$

donde $z_{1-\alpha/2} \approx 1.96$, $z_{1-\beta} \approx 0.84$, y δ es el MDE en unidades absolutas de revenue por sesión.

4. **Ajustar por estratificación y CUPED.** Si se aplica la técnica CUPED (*Controlled-experiment Using Pre-Experiment Data*) para reducir la varianza usando el revenue por sesión previo al experimento como covariable, el tamaño muestral efectivo requerido se reduce proporcionalmente a $1 - \rho^2$, donde ρ es la correlación entre la covariable pre-experimento y la métrica de experimento.
5. **Calcular la duración.** Dividir el tamaño muestral total ($2n$) entre el número de usuarios activos por semana para obtener la duración mínima del piloto. Se recomienda una duración de al menos dos semanas para capturar la variabilidad de días de la semana y amortiguar los efectos de novedad.

10.4 Guardrails y rollback automático

Los guardrails son condiciones de parada que, si se cruzan, desencadenan el rollback automático del tratamiento y la restauración de la condición de control para todos los usuarios del experimento. La lógica de monitorización debe ejecutarse con cadencia diaria durante las primeras fases del rollout y en tiempo cuasi-real durante la fase de mayor exposición.

10.4.1 Guardrail 1: caída de CTR del feed más allá de umbral

Si el CTR del feed en el tratamiento cae más de un Δ_{CTR} relativo respecto al control (umbral a calibrar con datos de tráfico real, típicamente en el rango del 10–15% relativo), el rollback se activa. Una caída de CTR de esta magnitud indica que el feed del tratamiento es percibido como menos relevante por el usuario, lo que —en el contexto del trade-off de F4— sugiere que la ponderación

$\lambda_{\text{rev}} = 0.5$ es excesiva para la sensibilidad de este segmento de usuarios. La acción de mitigación es retrogradar a $\lambda_{\text{rev}} = 0$ (relevancia pura) antes de considerar cualquier reexposición.

10.4.2 Guardrail 2: concentración de exposición de vendedores (índice de Gini)

Si el índice de Gini de exposición de vendedores en el feed del tratamiento supera un techo G_{max} (a establecer con datos de producción), el rollback se activa. Este guardrail protege la equidad de exposición: el ranking multi-objetivo, al ponderar revenue esperado, puede favorecer sistemáticamente a vendedores de ítems de mayor margen, concentrando la exposición y degradando la diversidad del catálogo percibida por el usuario. El objetivo f_{fair} del modelo F4 actúa como mitigación dentro del scorer, pero su efectividad real en producción debe verificarse.

10.4.3 Guardrail 3: latencia p99 del feed por encima del SLA

Si la latencia de extremo a extremo del feed en el percentil 99 supera el SLA acordado (referenciado en el sistema como p99), el rollback se activa. El pool de 200 candidatos y el reranker multi-objetivo aumentan el trabajo computacional por solicitud respecto al pipeline actual. El diseño de las cuatro fuentes del pool está optimizado para evitar llamadas adicionales al agregador externo (operan sobre tablas locales), pero la latencia total depende también del hardware de despliegue y de la carga concurrente. Si p99 supera el SLA, el sistema degrada a la condición de control antes de comprometer la experiencia de usuario con tiempos de respuesta inaceptables.

10.4.4 Guardrail 4: tasa de fallback del reranker LLM por encima de umbral

Si la fracción de solicitudes que activan el fallback del reranker LLM (descrito en los KPIs secundarios) supera un umbral τ_{fallback} —indicativo de que el modelo no puede completar el reranking para un porcentaje inaceptable de usuarios—, el rollback se activa. Este guardrail protege tanto la experiencia de usuario (el fallback produce un feed de menor calidad) como el costo operativo (un reranker que falla repetidamente e intenta reenvíos incrementa el consumo de tokens del modelo de lenguaje).

10.4.5 Guardrail 5: costo del agregador por compra fuera de presupuesto

En el e-commerce reseller objeto de estudio, cada llamada al agregador de catálogo (Amazon/AliExpress) tiene costo real. Si el costo agregado por compra completada en el tratamiento supera el presupuesto operativo acordado —ya sea por un aumento en el número de solicitudes de detalle de producto o por una reducción en la tasa de conversión que eleva el costo por compra—, el rollback se activa. Este guardrail es específico del modelo de negocio reseller y no tiene análogo directo en sistemas de recomendación de catálogo propio.

10.5 Riesgos y mitigaciones

10.5.1 R1: el trade-off de F4 daña el engagement a medio plazo

Riesgo. El estudio offline muestra que el punto knee (cfg8) obtiene +63.3% de **revenue@10** a costa de -59.8% en **nDCG@10** respecto al baseline RRF. Si esta caída de relevancia se traduce en un deterioro del engagement sostenido —menor CTR, menor profundidad de sesión, menor retención de usuarios a semanas vista—, el incremento de revenue a corto plazo puede revertirse a medio plazo por la pérdida de base de usuarios activos.

Mitigación. La estrategia de producción es conservadora por diseño: comenzar en el knee ($\lambda_{\text{rev}} = 0.5$) y monitorizar el engagement semana a semana durante el piloto. El ajuste hacia revenue mayor se realiza sólo si los indicadores de engagement (CTR, profundidad de sesión, tasa de retorno de usuarios) se mantienen estables o mejoran. Si el piloto confirma que el engagement aguanta, el ajuste posterior de λ puede delegarse a un bandit contextual que optimice por usuario, personalizando el trade-off en lugar de aplicar un peso uniforme a toda la base. En ningún caso se debe mover λ hacia el extremo de revenue máximo (cf^g18) sin evidencia explícita de que el engagement no se degrada.

10.5.2 R2: el detector de regalo introduce ruido en producción

Riesgo. El detector heurístico de sesiones de regalo alcanza una precisión de ≈ 0.43 en el estudio offline, lo que significa que activa el vector efímero de destinatario en aproximadamente el doble de sesiones que lo necesitan realmente. En producción, las sesiones de autoconsumo erróneamente clasificadas como regalo recibirán un feed orientado a un destinatario inexistente, con potencial daño en el CTR y la conversión de ese segmento.

Mitigación. El sistema implementa degradación graceful: ante un score de confianza de detección inferior a un umbral configurable, el pipeline opera en modo **self** en lugar de activar el vector efímero. El umbral se inicializa de forma conservadora (alta precisión, menor recall) y puede relajarse a medida que el piloto proporciona datos de calibración real. El guardrail de CTR del feed actuará como señal temprana si el detector introduce suficiente ruido como para dañar el engagement.

10.5.3 R3: validez externa limitada — sintético $\neq$ real

Riesgo. Los resultados cuantitativos del programa offline (tasas de recall, valores de nDCG@10, incrementos de revenue@10) fueron medidos sobre datos sintéticos con distribuciones de popularidad y patrones de co-ocurrencia controlados. Las distribuciones reales de comportamiento de usuario pueden diferir cualitativamente: la cola larga puede ser más larga, los patrones de regalo pueden ser estacionales, y la distribución de revenue por sesión puede tener una asimetría más severa que la simulada.

Mitigación. El piloto A/B es precisamente la validación externa que falta. Los resultados offline se utilizan únicamente para motivar el diseño experimental y calibrar el MDE, no para predecir resultados en producción. Las conclusiones cualitativas del programa —la multi-vectorialidad ayuda en sesiones de regalo, el pool multi-fuente amplía el recall, el trade-off relevancia-revenue es real y graduable— son las hipótesis que el piloto pone a prueba; sus valores cuantitativos específicos no se asumen como predicción.

10.6 Rollout por fases

El despliegue del tratamiento se realiza en cuatro fases secuenciales. El avance entre fases requiere la verificación explícita de que ningún guardrail ha sido cruzado durante la fase anterior.

10.6.1 Fase 0: modo sombra (*shadow mode*)

El pipeline de tratamiento se despliega en paralelo al de control, computando las listas de candidatos, el pool multi-fuente y el reranking multi-objetivo para todos los usuarios, pero **sin servir los resultados al usuario final**. Los resultados del tratamiento se registran en un log de sombra para análisis offline.

El objetivo de esta fase es verificar:

- Que el pipeline completo (FF-1 a FF-4 activados) se ejecuta sin errores de producción en todos los segmentos de usuario.
- Que la latencia **p99** del tratamiento se mantiene dentro del SLA antes de exponer al usuario real.
- Que la tasa de fallback del reranker LLM es inferior al umbral τ_{fallback} .
- Que el costo de las fuentes adicionales del pool (NPMI, popularidad por cohorte) no genera llamadas no previstas al agregador externo.

La fase sombra no requiere usuarios de tratamiento y puede ejecutarse sobre el 100% del tráfico existente. Su duración mínima es de 48 horas en condiciones de tráfico representativas.

10.6.2 Fase 1: exposición al 5% de usuarios

Los feature flags se activan para el 5% de la base de usuarios, asignados por hash determinista. El restante 95% permanece en control.

En esta fase se verifican los guardrails de CTR, Gini y **p99** con mayor sensibilidad (umbral de rollback más conservador) para detectar problemas sistémicos con mínima exposición. La duración mínima es de una semana para capturar la variabilidad de días laborables vs fin de semana. Al final de la semana, se realiza una revisión de los cinco guardrails antes de proceder a la siguiente fase.

10.6.3 Fase 2: exposición al 25% de usuarios

Si la fase 1 concluye sin guardrails cruzados, la exposición se eleva al 25% de usuarios. En esta fase el tamaño muestral comienza a ser suficiente para detectar efectos de magnitud moderada en la métrica primaria de revenue por sesión, lo que permite una primera evaluación estadística provisional.

La duración mínima es de dos semanas. Al final de esta fase se realiza el cálculo de potencia retrospectivo: si el efecto observado ya supera el MDE definido con potencia $1 - \beta \geq 0.80$, el equipo puede decidir acelerar el rollout; si el efecto es negativo y estadísticamente significativo, el rollback se activa.

10.6.4 Fase 3: rollout al 100% de usuarios

Si la fase 2 concluye sin guardrails cruzados y con un efecto no negativo en la métrica primaria, el tratamiento se despliega al 100% de la base de usuarios. En este punto los feature flags dejan de ser flags de experimento y pasan a ser la configuración de producción por defecto.

El período de observación post-rollout completo tiene una duración mínima de dos semanas adicionales para verificar la estabilidad de los indicadores a medida que el volumen de tráfico en el tratamiento alcanza su máximo. La infraestructura de guardrails permanece activa durante todo este período.

10.7 Resumen del protocolo

La tabla siguiente recoge los elementos esenciales del diseño del piloto en formato compacto para referencia rápida.

Elemento	Valor o criterio
Unidad de asignación	Usuario (hash determinista)
Condición de control	Pipeline actual (single-vector, top-30, RRF-only)
Condición de tratamiento	F1–F4: e2_hybrid + multi-vector + pool-200 + knee λ
Métrica primaria	Revenue / GMV por sesión
Nivel de significación	$\alpha = 0.05$ (bilateral)
Potencia objetivo	$1 - \beta = 0.80$
Estratificación	self vs gift (pre-asignación)
Duración mínima por fase	48 h (sombra), 1 semana (5%), 2 semanas (25%)
Guardrails de rollback	CTR, Gini vendedores, p99 latencia, fallback LLM, costo por compra
Punto de operación F4	knee min-max cfg8 ($\lambda_{\text{rel}} = 1$, $\lambda_{\text{rev}} = 0.5$)
Ajuste posterior de λ	Bandit contextual, sólo si engagement aguanta